

**Non-Profit JSC “L.N.Gumilyov Eurasian National University”
Institute of Digital Sciences and Artificial Intelligence
Department of “Computer and Software Engineering”**

EDUCATIONAL-METHODICAL COMPLEX OF DISCIPLINE

CSSE 33106 - Software Development
(code and name of the module)

on discipline DBT 3308 - ObjectRelational Database Management System
(code and name of the discipline)

6B06104 – “Computer Engineering and Software”
(code and name of the educational program)

**Non-Profit Joint-Stock Company «L.N. Gumilyov Eurasian National University»
«Institute of Digital Sciences and Artificial Intelligence»
Department Computer and software engineering education department**

Syllabus

CSSE 33106 - Software Development

on the discipline
DBT 3308 - ObjectRelational Database Management System for students of education program
6B06104 - Computer Engineering and Software

Астана
2026



This document, in accordance with paragraph 1 of Article 7 of the Law of the Republic of Kazakhstan dated January 7, 2003 «On electronic document and electronic digital signature» is equivalent to a document on paper

Document signed

№	Full name	Position
1	Senkovskaya Anastasiya	senior lecturer
2	Dyussekeyev Kanagat	Head of the кафедра
3	Davletova Ainash	Associate Professor (Associate Professor)
4	Orazaliyev Bakhytzhan	Director
5	Seilov Shakhmaran	Director of the Institute

Developer:

Сеньковская Анастасия Александровна, senior lecturer

Working curriculum (Syllabus) for the discipline DBT 3308 – Object-Relational Database Management Systems is developed on the basis of the educational program 6B06104 - "Computer Engineering and Software"

Considered at the meeting of the department of Computer and Software Engineering protocol # 10 from “04” May 2026 y.

Approved at the meeting of the Institute’s Teaching and Methodical Commission Protocol # 9 from “14” May 2026 y.

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
---	---	-------------------------------	----------------

EXPLANATORY NOTE

1. Brief description of the discipline

The course studies the theoretical foundations of building databases (DB), basic operations on data, methods of organizing data search and processing, language tools for describing and manipulating data. The principles of building basic data models and their use in modern database management systems (DBMS) are considered. Special attention is paid to languages and database processing facilities. Object-relational data mapping (ORM) is considered. Modern object-relational databases and software tools for working with them are studied.

Aim	Learning outcomes (LO) according to the educational program*	Expected learning outcomes (LO) in the discipline
Developing skills and abilities in installing and configuring distributions (software); Learning SQL and XQuery query languages; Query optimization; Mastering commands and methods for manipulating data in modern database management systems.	RT₁₀ - To apply the theoretical knowledge, practical skills and creative approach in the field of advanced technologies for developing modern interactive applications, databases, realize the synthesis of stereoscopic images, use tracking technologies and game engines, demonstrate programming technologies for implementing multi-threading (multitasking) software; PC₆ – Ability to form knowledge and skills in the field of digital technologies and computer systems (used software tool environments and tools for creating digital signal processing systems, SMART systems, robotic complexes)	To know: install and configure Microsoft SQL Server; perform infological design of databases using the "entity-relationship" method. To be able: to perform logical design of relational databases; normalize database interaction schemes; create and destroy databases, manage their volume; modern devices of computing systems and networks of work rules; conduct experimental research; be able to process the results of experiments.

* According to EP 6B06104 - "Computer Engineering and Software" 2024 y

2. Prerequisites

Knowledge, skills and abilities acquired during the study of the following disciplines are necessary to master this discipline: Algorithms and Data Structures.

Postrequisites

The knowledge, skills and abilities acquired during the study of the discipline are necessary for the learning of the following disciplines: Web Application Development, Diploma engineering.

3. Extract from the curriculum

Year	3
Semester	5
Number of ECTS	7

Types of classes	Total number of hours
Lectures	30
Practical classes	15
Seminars	-
Laboratory practicals	30
Independent work of a student (IWS)	135
Total	210

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
---	---	-------------------------------	----------------

4. Thematic plan of the discipline by modules
(in academic hours)

Module №	Name of the module
1	Introduction to Data Management
2	Learning SQL, XQuery languages

Lectures				
Week №	Module №	The theme of the lecture	Number of hours	Types and methods of training
1	1	Introduction. Course Subject. Understanding Data. Topic Questions: Basic Definitions	2	Presentation, discussion explanatory and illustrative method.
2	1	Three Basic Data Models. Relational Databases. Questions on the Topic: Genealogy of High-Level Programming Languages.	2	Presentation, discussion explanatory and illustrative method.
3	1	Stages of designing relational databases. Questions on the topic: modular and structured programming.	2	Presentation, discussion explanatory and illustrative method.
4	1	SQL (Structured query language), basic SQL commands. Questions on the topic: types of abstractions and program preparation.	2	Presentation, discussion explanatory and illustrative method.
5	1	Single row functions. Questions on the topic: standards and life cycle models.	2	Presentation, discussion explanatory and illustrative method.
6	1	Getting data from multiple tables. Joins. Questions on the topic: mathematical description of the solution to the problem.	2	Presentation, discussion explanatory and illustrative method.
7	1	Summarizing data using group functions. Questions on the topic: methodology, SADT and decomposition.	2	Presentation, discussion explanatory and illustrative method.
8	2	Queries. Related subqueries. Hierarchical queries. Questions on the topic: task structure and hierarchy.	2	Presentation, discussion explanatory and illustrative method.
9	2	Complex questions. Transposing tables using PIVOT. Questions on the topic: abstraction, encapsulation, polymorphism, inheritance.	2	Presentation, discussion explanatory and illustrative method.
10	2	Data Manipulation (DML) Questions on Topic: UML Diagram.	2	Presentation, discussion explanatory and illustrative method.
11	2	Creating and Managing Tables (DDL). Questions on the topic: Logic and Principles.	2	Presentation, discussion explanatory and illustrative method.
12	2	Relational DB normalization: 1nf, 2NF, 3nf. Topic questions: flexibility, scalability, reuse and security.	2	Presentation, discussion explanatory and illustrative method.

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
---	---	-------------------------------	----------------

13	2	Database objects (view, sequence, index, synonym). Questions on the topic: logical elements, components in hardware nodes.	2	Presentation, discussion explanatory and illustrative method.
14	2	Division. (Distribution), demand cost, demand optimization, indexation. Questions on the topic: Black-Box, white box, Grey Box.	2	Presentation, discussion explanatory and illustrative method.
15	2	OLAP. DWH, ETL. Other relational DBMS. Questions on the topic: spiral testing process.	2	Presentation, discussion explanatory and illustrative method.
TOTAL			30	

Practical classes				
Week №	Module №	The theme of the Practical classes	Number of hours	Types and methods of training
1	1	Database architectures	1	Report
2	1	Database Management	1	Report
3	1	Database design	1	Essay
4	1	Hierarchical data model	1	Report
5	1	Conceptual model of a database	1	Essay
6	1	Basic concepts of database systems	1	Report
7	1	Scaling Distributed Databases	1	Essay
8	2	Data flow management	1	Report
9	2	Data warehouses	1	Report
10	2	Distributed and parallel databases	1	Report
11	2	SQL language	1	Report
12	2	XQuery language	1	Report
13	2	NoSQL language	1	Report
14	2	OLAP technologies	1	Essay
15	2	Types of modern databases	1	Essay
TOTAL			15	

Laboratory classes				
Week №	Module №	The theme of the Laboratory classes	Number of hours	Types and methods of training
1	1	Introduction to SQL. Distributions. Installing Microsoft SQL Server. Objective of the lesson: Installing and configuring Microsoft SQL Server.	2	Instruction
2	1	Introduction to SQL. Components and interface of Microsoft SQL Server. Objective of the lesson: To learn the interface of Microsoft SQL Server.	2	Completing laboratory work
3	1	Creating a database. SQL commands. Working with tables. Purpose of the work: To become familiar with table terminology and write code.	2	Completing laboratory work

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
---	---	-------------------------------	----------------

4	1	Database structure objects. Purpose of the work: Syntax rules and basic queries. Objective: to write a program according to the version	2	Individual tasks
5	1	Introduction to Transact-SQL. Purpose of the work: apply Transact-SQL.	2	Completing laboratory work
6	1	Data selection from tables. Purpose of the work: Learn to make calculations and computations, as well as convert data. Write a program.	2	Individual tasks
7	1	Stored procedures. Purpose of the work: to study the aim and application of stored procedures.	2	Completing laboratory work
8	2	Security in Microsoft SQL Server Purpose of the work: to study the roles of database users and be able to assign them. Credentials and backup.	2	Individual tasks
9	2	Database management. Purpose of the work: Write a program. Basic functions -Permission.	2	Completing laboratory work
10	2	Selecting data from several tables. Purpose of the work: To be able to perform various manipulations: rename, sort, set a range, use special characters and other types of table joins.	2	Individual tasks
11	2	Analytical data sampling. Purpose of the work: Learn to identify project risks.	2	Individual tasks
12	2	Apply Views. Purpose of the work: View assignment, filters and creation.	2	Individual tasks
13	2	Database design Purpose of the work: To become familiar with the main tools and software for database design (Erwin Studio, etc.)	2	Completing laboratory work
14	2	Visual representation. Purpose of the work: Modeling data in the form of UML diagrams.	2	Individual tasks
15	2	Data management. Purpose of the work: Introduction to the object-oriented .Net platform	2	Completing laboratory work
TOTAL			30	

Independent work of students				
Week №	Module №	The theme of the independent work of students	Number of hours	Types and methods of training
1	1	Database architectures	9	Report
2	1	Types of Database Management	9	Report
3	1	Database design	9	Essay
4	1	Hierarchical data model	9	Report
5	1	Conceptual model of a database	9	Essay

		«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
6	1	Basic concepts of database systems	9	Report
7	1	Scaling Distributed Databases	9	Essay
8	2	Data flow management	9	Report
9	2	Data warehouses	9	Report
10	2	Distributed and parallel databases	9	Report
11	2	SQL language	9	Report
12	2	XQuery language	9	Report
13	2	NoSQL language	9	Report
14	2	OLAP technologies	9	Essay
15	2	Types of modern databases	9	Essay
TOTAL			135	

5. Brief organizational and methodological characteristics of the discipline

Types of control of educational achievements:

Routine control 1 colloquium

Routine control 2 colloquium

Final control: oral exam

Course Policies and Procedures

- do not be late for classes;
- do not miss classes without a good reason;
- preliminary preparation for classes;
- timely execution and delivery of IWS;
- preparation for all types of classes should be independent and creative;
- creating an inclusive and equal learning environment where every student feels valued and

respected. The learning process takes into account the individual characteristics of each student, including differences in abilities, learning styles, cultural and social background.

6. Educational and methodological coverage of the discipline

№	Author, name, publisher, year of publication	Information carrier	Quantity (pieces)		in the department
			in the library	World Digital Libraries	
1	2	3	4	5	6
Basic literature					
1	Novikov B.A. Fundamentals of database technologies: a tutorial / B.A. Novikov, E.A. Gorshkova, N.G. Grafeeva, edited by E.V. Rogov. - 2 nd ed. - Moscow: DMK Press, 2020. – 582 p. ISBN 978-5-97060-841-8.	Electronic textbook		https://edu.postgrespro.ru/dbtech.pdf	+
2	D.L. Osipov. Technologies database design. – M.: DMK Press, 2019. - 498 p.: ill. ISBN 978-5-97060-737-4	Electronic textbook		https://ftp.zhiron.kz/books/IT/Базы данных/Технологии проектирования баз данных (Д.Л. Осипов).pdf	+

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
---	---	-------------------------------	----------------

3	Bekmanova G.T. Databases: a textbook for master's students specialty "6M060200 Computer Science" / G.T. Bekmanova. Astana: Master Po, 2017. - 71, [1] p: table. Bibliography: p. 71. - ISBN 978- 601-326-009-9.	Study guide	1		-
Additional literature					
4	Konkov K.A. The structure and functioning of the Windows OS / K.A. Konkov. - Moscow: National Open University INTUIT, 2024. - 239 p. - ISBN 978-5-94774-827-7.	Electronic textbook		https://www.ibooks.ru/bookshelf/394576/readings	4
Electronic and Internet resources					
1	Rodrigues F., Coles M., Dye D. Pro SQL Server 2012 Integration Services, Apress, 2012. – 636 pp. https://dl.nemoudar.com/Pro-SQL-Server-2012-Integration-Services-Nemoudar.pdf				

7. System of learning outcomes assessment of a student

Students' knowledge, skills and abilities are evaluated according to the following system

Grade by letter system	Digital equivalent of grade	Percentage content	Grade by traditional system	Grade criteria
A	4,0	95-100	Excellent	The A grade is given when a student demonstrates mastery in: installing/configuring MS SQL Server; complex SQL/XQuery queries with optimization; advanced normalization (1NF-3NF+); OLAP/ETL processes; precise UML/ER modeling; object- relational mapping (ORM). Independently solves complex problems (e.g., query transposition with PIVOT, stored procedures, security roles), justifies decisions logically, consistently and competently, not have difficulty answering when changing the type of task, freely answered questions, demonstrated knowledge of monographic materials, could correctly justify the decisions made, used complex skills and techniques when performing practical work, if he showed that he can accurately formulate the material and can present it.
A-	3,67	90-94		The A- grade is given when a student demonstrates mastery in: installing/configuring MS SQL Server; complex SQL/XQuery queries with optimization; advanced normalization (1NF-3NF+); OLAP/ETL processes; precise UML/ER modeling; object- relational mapping (ORM). Independently solves complex problems (e.g., query transposition with PIVOT, stored procedures, security roles), justifies decisions logically, consistently and competently, not have difficulty answering when changing the type of task, freely answered questions, demonstrated knowledge of monographic materials, could correctly justify the decisions made, used complex skills and techniques when performing practical work, if he showed that he can accurately formulate the material and can present it. There may be shortcomings in the definition of concepts, corrected by the student himself in the process of answering.

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
---	---	-------------------------------	----------------

B+	3,33	85-89	Good	The B+ grade is given when a student is proficient in core skills: DB design (conceptual/logical); SQL joins/group functions; basic normalization; managing tables/views. Can optimize simple queries and implement security roles. Lab work is thorough but may lack advanced polish (e.g., hierarchical queries). The answer is clearly structured, logical, presented in literary language in terms of science. There are shortcomings in the answer, corrected by the student with the help of the teacher.
B	3,0	80-84		The B grade is given when a student is competent in fundamental tasks: creating databases/tables; single-table queries; basic DML/DDDL commands. Understands normalization principles but applies inconsistently. Lab work completed with minor errors (e.g., syntax issues in stored procedures). The answer is clearly structured, logical, presented in literary language in terms of science. There may be shortcomings or minor errors corrected by the student with the help of the teacher.
B-	2,67	75-79		The B- grade is given when a student is partial proficiency: Can install SQL Server and write basic queries (SELECT/INSERT), but struggles with multi-table joins, subqueries, or normalization. Limited ability in security/backup tasks. Oral answers reveal gaps in terminology (e.g., confusing OLAP vs. OLTP). The answer is clearly structured, logical, stated in terms of science. However, minor mistakes or shortcomings were made, corrected by the student with the help of guiding questions.
C+	2,33	70-74		The C+ grade is given when a student has a basic understanding: completes simple labs (e.g., table creation) but makes significant errors in query logic or ER modeling. Cannot optimize/index queries. Complete but insufficiently consistent answer to the question is given, but at the same time the ability to identify essential and non-essential signs and cause-and-effect relationships is shown. The answer is logical and stated in terms of science. There may be 1-2 mistakes in the definition of basic concepts that the student found it difficult to correct on their own.
C	2,0	65-69	Satisfactory	The C grade is given when a student has inconsistent performance: can describe DB architectures/hierarchical models theoretically, but cannot apply them (e.g., fails to design a 1NF schema). Limited SQL syntax knowledge. The logic and sequence of the presentation have violations. Mistakes were made in the disclosure of concepts, the use of terms. The student is not able to independently identify essential and non-essential signs and cause-and-effect relationships. The student can concretize generalized knowledge by proving their main points by examples only with the help of a teacher. Speech design requires corrections.
C-	1,67	60-64		The C- grade is given in the case when an incomplete answer is given, the logic and sequence of presentation have significant violations. Gross mistakes were made in determining the essence of the disclosed concepts, theories, phenomena, due to students' misunderstanding of their essential and non-essential features and connections. There are no conclusions in the response. The ability to reveal specific manifestations of generalized knowledge is not shown. Speech design requires corrections.

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
---	---	-------------------------------	----------------

D+	1,33	55-59		The D+ grade is given when an incomplete answer is given. There is an illogical presentation. The teacher finds it difficult to prove. There are a lot of significant errors in the definitions of terms, concepts, characteristics of facts, phenomena. There are no inputs in the response. Speech is illiterate. When answering additional questions, the student begins to realize the existence of a connection between knowledge only after the teacher prompts.
D	1,0	50-54		The D grade is given when an incomplete answer is given, which represents scattered knowledge on the topic of the question with significant errors in definitions. There is fragmentary, illogical presentation. The student is not aware of the connection of this concept, theory, phenomenon with other objects of the module (discipline). There are no conclusions, concretization and evidence- based presentation. The speech is illiterate. Additional and clarifying questions from the teacher do not lead to correction of the student's answer not only to the question posed, but also to other questions of the module (discipline).
FX	0,5	25-49	Unsatisfactory	The grade “unsatisfactory” corresponds to the letter FX , F , which has a digital equivalent of 0 and a percentage of 0-49. This assessment is made if the student has found gaps in the knowledge of the basic material provided by the program, has not mastered more than half of the module (discipline) program, has made fundamental mistakes in the answers, has not completed individual tasks provided for by the forms of current, intermediate and final control, has not worked through all the basic literature provided by the program.
F	0	0-24		

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

LECTURE ABSTRACTS

Lecture No. 1.

Topic: Introduction. Course Subject. Understanding Data. Questions on Topic: Basic Definitions

The purpose of the lecture: gain an understanding of the current state of software; study the basic concepts and approaches to programming technology; gain an understanding of structured, modular, object and visual programming.

Relational data is data from relational databases (tables). Multidimensional data is data presented in OLAP cubes. A dimension or axis – in multidimensional data – is a set of data of one type that allows creating multidimensional databases. During the solution of the problem, data corresponding to the criterion of stability of their values can be:

- * variables;
- * stable;
- * conditionally stable.

Variable data are data whose values change during the problem solving process. Constant data are data that retain their values during the problem solving process (mathematical constants, coordinates of stationary objects) and do not depend on external factors. Conditional constant data are data that can sometimes change their values, but these changes are determined by external factors and do not depend on the problem solving process.

Depending on the functions they perform, information can be reference, operational, or archival. You must distinguish between period data and point data. These distinctions are important both when designing a purchasing system and in the measurement process:

- * data by period;
- * point data. Periodic data describes a specific period of time. Examples of periodic data include: business profit for a month, average temperature for a month. Point data represents the value of a variable at a specific time. Examples of point data include: account balance on the first day of the month, temperature at eight in the morning.

Data is primary and secondary. Secondary data is data that is the result of certain calculations applied to the original data. Secondary data usually results in an instant response to the user's request by increasing the amount of information stored.

Literature

1. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
2. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 2.

Topic: Three Basic Data Models. Relational Databases. Questions on the Topic: Genealogy of High-Level Programming Languages.

The purpose of the lecture: genealogy of high-level programming languages. The relational data model is a logical data model developed

Edgar Codd, who describes:

- * a data structure in the form of a set of relations (changing over time);
- * aggregate-theoretical operations with data: union, intersection, difference and cartesian product;
- * special relational operations: selection, projection, union and division;
- * special rules that ensure data integrity.

Edgar Frank "Ted" Codd (August 23, 1923 – April 18, 2003) was a British computer scientist whose work formed the basis of relational database theory. While working at IBM, he created the relational model

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

of data. In 1970, he published The Relational Data Model for Large Shared Data Banks, which is considered the first work on the relational data model. The relational data model is a way of looking at data, that is, a prescription for how to represent data (through tables) and how to work with that representation (through statements). It addresses three aspects of data: structure (objects), data integrity, and processing (statements).

The goals of building a relational data model:

- * ensuring a high degree of data dependence;
 - * creating a robust foundation for addressing semantic issues and data consistency and redundancy;
 - * extending data manipulation languages by adding operations to sets.
- Relational databases support relationships that change over time.

Literature

1. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
2. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 3.

Topic: Stages of designing relational databases. Questions on the topic: modular and structured programming

The purpose of the lecture: SQL language.

The SQL database interaction language emerged in the mid-1970s, and an experimental relational DBMS system was developed within the R project. The original name of the language, SEQUEL (Structured English Query Language), only partially reflects the essence of this language. Of course, this language was mainly focused on creating queries to relational databases that were convenient and understandable for users, but in fact it was a complete database that included tools for formulating queries and defining and manipulating the database schema in addition to operators for managing databases; defining integrity constraints and triggers; database views; a physical representation that supports efficient query execution; the ability to define level structures; confirming access to relations and their fields; saving and returning transactions. The means of synchronizing access to data objects in the language from the side of parallel transactions allow.

As you know, the two main languages for querying relational databases are relational algebra and relational computing languages. Despite this rigorous and theoretical foundation, these languages are rarely used as user interface tools in modern relational DBMSs. Queries in these languages are difficult to formulate and understand. SQL is a combination of relational tuples and relational algebra, and there is still no general agreement on which of the classical languages is closest. In addition, the capabilities of SQL are broader than those of these main relational languages, in particular, in general, a query formulated in SQL cannot be translated into a relational algebra expression, which requires some extension.

Important properties of the internal SQL query language are the ability to easily formulate queries with the addition of multiple relations and the use of internal subqueries in selection predicates. In general, having both tools at the same time is redundant, but it gives the user the ability to choose the more understandable option when formulating a query. Subquery predicates in SQL System R can use the comparison operators established by theoreticians, allowing numeric queries to be formulated (these features are usually the most difficult for users to understand, which is why explicit numeric predicates were introduced later in SQL).

Test questions:

- 1) What is the purpose of comma in SQL queries?
- 2) how to view table structure?

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

3) What is the purpose of SQL index?

Literature

1. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
2. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 4.

Topic: SQL (Structured query language), basic SQL commands. Questions on the topic: types of abstractions and program preparation

Basic SQL commands are not limited to the standard CREATE, UPDATE, and DELETE. This article will be useful for those who want to improve their SQL knowledge before an interview. Note that the examples use MySQL, so each statement ends with a semicolon. Before you begin, create a database with test data. To work, you need to download two files: DLL.sql and InsertStatements.sql. Once MySQL is installed, open a terminal and enter it into the MySQL console using the following command: `mysql -u root -p`

Then enter the password and run the following command. Let's call the database "university":

```
CREATE
DATABASE university;
USE university;
;
SOURCE <path_of_DLL.sql_file>;
SHOW DATABASES
```

Please use the SQL commands.

CREATE DATABASE

Command to create a new database.

USE

USE <database_name> SQL the team selects the database needed for further work with it.

SOURCE

ASOURCE <file.sql> allows you to execute multiple SQL commands at once in a file with the extension .sql.

DROP DATABASE

SQL command is presented with each other SHOW TABLES

With this simple command you can see all the tables available in the database. CREATE TABLE

SQL command to create a new table:

```
CREATE TABLE <table_name1> (
    <col_name1> <col_type1>,
    <col_name2> <col_type2>,
    <col_name3> <col_type3> PRIMARY
    KEY (<col_name1>),
    FOREIGN KEY (<col_name2>) REFERENCES <table_name2> (<col_name2>)
);
```

CREATE TABLE integrity constraints when used. You may need to create constraints on specific columns in a table. When creating a table, you can set the following constraints:

- table cell cannot have a value NULL;

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

- primary key -PRIMARY KEY(col_name1, col_name2, ...);
- foreign key -FOREIGN KEY(col_namex1, ..., col_namexn) REFERENCES table_name(col_namex1, ..., col_namexn).

You can specify multiple primary keys. In this case, you will get a composite primary key.

Example: Scheduling "instructor":

CREATE TABLE instructor(

```

IDCHAR(5),
nameVARCHAR(20)NOTNULL,
dept_nameVARCHAR(20), salary
yNUMERIC(8,2), PRIMARY
KEY(ID),
FOREIGN KEY(dept_name)REFERENCES department(dept_name)
);

```

DESCRIBE

DESCRIBE <table_name> displays various information (value type, key or not) about the table columns. SQL is most often used to create loads, data (with report assembly) and database management.

Test questions:

- 1) How to create keys?
- 2) What are links used for?
- 3) Give examples with basic SQL commands?

Literature

1. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
2. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No.5.

Topic: Single row functions. Questions on the topic: standards and life cycle models.

The purpose of the lecture: gain an understanding of the composition of processes and the evolution of life cycle models, and consider the impact of CASE technology on changes in the software life cycle.

The functions used in an SQL query can be divided into two large groups:

- functions that assign a single value to the data of a single row when selected (single-row functions).
- functions that give one value for several data series (multi-row functions).

Any function in SQL, whether single-row or multi-row, is designed to accept (or reject, with parameterless functions) parameters and return only one value of a specific type at a time.

Let's dive a little deeper. Regarding

"one and only value" returned by a function, then remember that you can set OUT parameters for PL/SQL functions, which allows the function to return multiple values. This refers to the value returned directly by the function, not via parameters. Also, we are not studying PL/SQL at the moment, that is a topic for another exam. Let's look at SQL and functions that can be used in the SELECT statement, while you cannot use functions with OUT parameters in SELECT.

A function may have no parameters or it may have one or more parameters. The settings may vary.

When calling a function, the parameters must be separated by commas. A function can be represented as pseudocode: result = F (param1, param2, param3, ...) When you call a function as a parameter, you can specify another function, which may have its own parameters. For example, for example: result = F1 (param1, param2, F2 (p1, p2, ...), param4, ...) In most cases, in a SQL query, we will need to use functions to work with data such as "string", "number" and "date". String functions

String functions are functions for processing string data. Functions are needed to return string data (for example, to get the length of a string) or to transform the original string and return another string value after processing the input string. At this point, we are interested in built-in functions (although you can create your own functions and use them in queries, but this is a separate topic that is not covered in this section).

Test questions:

- 1) What does the word relationship mean in relation to relational databases?
- 2) how to view table structure?
- 3) SQL commands that initiate and end a transaction?

Literature

1. Allen, G. Taylor SQL for dummies / Allen G. Taylor. - M.: Dialectics, Williams, 2015. - 416 p.
2. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
3. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 6.

Topic: Getting data from multiple tables. Joins. subtopics: join tables

The SQL JOIN operator is designed to combine two or more database tables according to the corresponding condition. This operator exists only in relational databases. It is thanks to JOIN that a relational database has such powerful functionality that allows not only storing data, but also analyzing it using queries, at least in a simple way. Let's look at the main nuances of writing SQL queries with a common JOIN operator for all DBMS (database management systems). The SQL JOIN operator has the following syntax for combining two tables:

```
SELECT COLUMN_NAMES (1..N) FROM TABLE_NAME_1 JOIN TABLE_NAME_2
ON CONDITION
```

You can continue one or more links with the JOIN operator using a clause that specifies the selection condition, such as in a simple SELECT query, with an optional where or HAVING. What is common to all DBMS is that they can appear in the assembly instead of JOIN, INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, FULL OUTER JOIN, CROSS JOIN (or,

alternatively, a comma). A query with the INNER JOIN operator is designed to join tables and display the resulting table in which the data completely intersects according to the condition specified after the ON clause.



JOIN just does that. So the word INNER is optional.

Literature

1. Allen, G. Taylor SQL for dummies / Allen G. Taylor. - M.: Dialectics, Williams, 2015. - 416 p.
2. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
3. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 7.

Topic: Summarizing data using group functions.

subtopics: methodology, SADT and decomposition

Aggregate functions exist to somehow generalize the data we receive, that is, to allow us to manipulate it the way we want. These functions are implemented using keywords included in the SELECT query, and their writing will be described later. For clarity, SQL provides some capabilities for aggregate functions:

- * Add selected values
- * Find the arithmetic mean of the values
- * Find the minimum and maximum values.

SQL Aggregate Function Examples We will look at the most commonly used functions and give some examples. SUM Function This function allows you to add field values to a SELECT query. Like all aggregate functions in SQL, the syntax is very simple and the function is very useful. Let's start with an example right away to understand:

Subtract the sum of all orders from the table of orders completed in 2016. The size of the orders could be canceled, but that's pretty easy. Let's recall the structure of our schedule:

onum	amt	odate	cnum	snum
1001	128	2016-01-01	9	4
1002	1800	2016-04-10	10	7
1003	348	2017-04-08	2	1
1004	500	2016-06-07	3	3
1005	499	2017-12-04	5	4
1006	320	2016-03-03	5	4
1007	80	2017-09-02	7	1
1008	780	2016-03-07	1	3
1009	560	2017-10-07	3	7
1010	900	2016-01-08	6	8

The following code performs the necessary selection:

```
SELECT SUM (amt)
FROM Orders
WHERE odate BETWEEN '2016-01-01' and '2016-12-31';
```

As a result we get:

SUM(amt)
4428

In this query, we have used the SUM function and then the collection field must be specified in brackets. Then we have defined a condition in the WHERE that selects rows only from 2016. Actually, this condition can be written differently, but now the sum function in SQL is important. AVG Function The following function calculates the arithmetic mean of the data field that we specify as a parameter. The syntax of this function is the same as the sum function. COUNT Function This function is needed to count the number of selected values or rows. There are two main ways to use it:

- with the DISTINCT keyword to count non-transitive values;
- Use the "*" sign to count all selected values Now let's look at an example of using COUNT in SQL: Count number of orders and number of sellers in the orders table.

```
SELECT COUNT(*), COUNT(DISTINCT snum)
FROM Orders;
```

As a result we get:

COUNT(*)	COUNT(snum)
10	5

Obviously, the number of orders is 10, but if you suddenly have a large table, this feature will be very convenient. Here you should use DISTINCT for unique sellers, because one seller can serve several orders.

Literature

1. Allen, G. Taylor SQL for dummies / Allen G. Taylor. - M.: Dialectics, Williams, 2015. - 416 p.
2. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
3. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 8.

Topic: Queries. Related subqueries. Hierarchical queries. Subtopics: types of requests

Typical methods of structured design: design, coding and testing of programs; modular programming. The top-down design method involves the sequential division of the general data processing function into simple functional elements ("top-down"). As a result, a hierarchical scheme is created, reflecting the composition and mutual subordination of individual functions, which is called the functional structure of the application algorithm (FSA). The sequence of actions for creating the functional structure of the application algorithm

- initially, the goals of automation of the subject area and their hierarchy (goal-subgoal) are defined;
- the composition of applications (processing tasks) that ensure the implementation of the set goals is established;
- the nature of the interrelations between applications and their main characteristics (information for solving problems, time and frequency of solution, conditions of execution, etc.) are specified);
- the data processing functions required to solve problems are defined;
- processing functions are decomposed to the required structural complexity, which is implemented by the tools provided for use. Each individual FSA element describes a full-fledged meaningful information processing function, which includes a specific method of implementation at the

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

software level. Modular Programming is based on the concept of a module - a logically interconnected set of functional elements, implemented in the form of individual software modules.

The module must have the following properties:

- one input and one output - at the input, the software module receives a certain set of initial data, processes the contents and returns one set of initial data, i.e. standard-input-process-output is performed;
- logical independence - the result of a software module depends only on the initial data, but not on the work of other modules;
- weak information links with other software modules - the exchange of information between modules should be kept to a minimum as much as possible;
- a software element visible in size and complexity. Thus, modules have schemes for data detection, data processing operations, and communication with other modules that can be processed. Each module consists of a specification and a body. The features define the rules for using the module, and the body defines how the processing is implemented.

Literature

1. Allen, G. Taylor SQL for dummies / Allen G. Taylor. - M.: Dialectics, Williams, 2015. - 416 p.
2. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
3. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 9.

Topic: Complex questions. Transposing tables using PIVOT.

subtopics: abstraction, encapsulation, polymorphism, inheritance

The relational operators PIVOT and UNPIVOT can be used to change an expression evaluated in a table into another table. PIVOT transforms the unique values of one column of the expression into multiple output columns and transforms the expression into a table. If necessary, PIVOT also merges the remaining duplicate column values and displays them in the result. UNPIVOT transforms the columns of the expression evaluated in a table into column values and performs the inverse of the PIVOT. Syntax PIVOT SELECT ... This is simpler and more straightforward than the syntax that can perform a single action using the order of CASE statements. The following syntax describes the use of the PIVOT operator.

SQL syntax

```
SELECT<non-pivoted column>,
[first pivoted column]AS<column name>, [second pivoted column]AS<column name>,
...
[last pivoted column]AS<column name>FROM (<SELECT query that produces the data>)AS<alias for the source query>
PIVOT (
<aggregation function>(<column being aggregated>)
FOR
[<column that contains the values that will become column headers>]IN( [first pivoted column],
[second pivoted column],
... [last pivoted column])
)AS<alias for the pivot table>
<optional ORDER BY clause>;
```

Basic PIVOT Example The following code example creates a table with two columns and four rows.

SQL

```
USEAdventureWorks2014 ; GO
SELECTDaysToManufacture,AVG(StandardCost)ASAverageCostFROMP roduction.Product
GROUP BYDaysToManufacture;
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

A common use of the PIVOT operator is to create crosstab reports to present summaries of data. For example, suppose you want to access the PurchaseOrderHeader table in the AdventureWorks2014 database to determine the number of purchase orders for a specific employee. The following query would return the data you need, sorted by supplier.

Literature

1. Allen, G. Taylor SQL for dummies / Allen G. Taylor. - M.: Dialectics, Williams, 2015. - 416 p.
2. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
3. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 10.

Topic: Data manipulation(DML).

Subtopics: teamsDML

MS SQL is very much related to the syntax of the DML SELECT construction in the DB dialect, and then I start talking about DML with it. In my opinion, the SELECT construction is the most important construction of the DML language, since depending on it or its parts, the selection of the necessary data from the database is carried out. DML contains the following structures:

- * SELECT-selecting data
- * INSERT - entering new data
- * UPDATE-Update data
- * DELETE - delete data
- * MERGE - data merging

In this section, we will only cover the basic syntax of the SELECT command, which looks like this:

Column List

SELECT [DISTINCT] or
FROM Source WHERE Filter
ORDER BY Sort by order

The topic of SELECT statement is quite vast, so in this section I will focus only on its basic structures. I don't think you can start learning complex structures without knowing the database well, because then everything will revolve around this basic structure (secondary questions, unions, etc.). Because the basic syntax is implemented differently in different dialects of SQL language. If DDL is static, that is. with its help rigid structures are created (tables, links, etc.). Then, DML language is dynamic in nature, where you can get correct results in different ways.

The training also continues in the "step by step" mode, i.e. when reading, you should immediately try to complete the example with your own hands. After that, analyze the result and try to understand it intuitively. If something becomes unclear, for example, the meaning of a function, then seek help on the Internet.

Test questions:

- 1) How are the three rules of the first normal form ...
- 2) What capabilities does SQL provide for examining query details?
- 3) What must be defined in two tables to join them?

Literature

1. Allen, G. Taylor SQL for dummies / Allen G. Taylor. - M.: Dialectics, Williams, 2015. - 416 p.
2. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
3. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 11.

Topic: Creating and managing tables (DDL).

Subtopics: Main characteristics of DDL

A database is a structured set of records or data stored in a computer system and systematized for quick retrieval and retrieval of the necessary information. Keep in mind that the following logic works in MS SQL:

- Integer / Integer = Integer (i.e. in this case integer division occurs);
- Real / Whole = Real;
- Integer / Fact = Actual;

That is, the result is transformed into a larger form, so in the last 2 cases we get a real number (think of it like in mathematics - the range of real numbers is larger than the range of integers, so the result is transformed into it):

SELECT
123/10,-- 12
123./10,-- 12.3
123/10.-- 12.3

In some applications, it is very important that the sequence of queries is in the correct order and that each individual query is executed. In addition, transactions allow multiple users or programs to access the database at the same time.

Test questions:

- 1) What commands are used to view available databases?
- 2) what is the difference between numeric and associative array?
- 3) what must be defined in two tables to join them?

Literature

1. Allen, G. Taylor SQL for dummies / Allen G. Taylor. - M.: Dialectics, Williams, 2015. - 416 p.
2. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
3. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 12.

Topic: Relational DB Normalization: 1NF, 2NF, 3NF.

SubTopics: flexibility, scalability, reusability and security.

The process of designing databases using the NF method is iterative and consists of successive transfer of the relationship from 1NF to NF according to certain rules. Each subsequent NF is limited by a certain type of functional dependence and the elimination of corresponding deviations when performing operations on DB relations, as well as preserving the properties of the previous NF.

First normal form A relation is 1nf, and if all its attributes are simple, all used domains must have only scalar values. There must be no duplicate rows in the table. For example, there is a table "cars":

Firm	Models
BMW	M5, X5M, M1
Nissan	GT-R

Violation of 1NF normalization occurs in BMW models, since one cell contains a list of 3 elements: M5, X5M, M1, i.e. it is not an atom. Let's transform the table into 1nf:

Firm	Models
BMW	M5
BMW	X5M
BMW	M1
Nissan	GT-R

Second Normal Form A relationship is in 2NF if it is in 1nf and every non-key attribute depends on the primary key (PK). Irreducibility means that a candidate key does not contain a subset of attributes from which this functional dependency can be extracted. For example, given the table:

<u>Model</u>	<u>Firm</u>	Price	Discount
M5	BMW	5500000	5%
X5M	BMW	6000000	5%
M1	BMW	2500000	5%
GT-R	Nissan	5000000	10%

The table is in first normal form, but not in second. The price of a car depends on the model and the brand. The discount depends on the brand, so the dependence on the primary key is not complete. These non-key attributes are corrected by decomposing into two relations that depend on the PK.

<u>Model</u>	<u>Firm</u>	Price
M5	BMW	5500000
X5M	BMW	6000000
M1	BMW	2500000
GT-R	Nissan	5000000

<u>Firm</u>	Discount
BMW	5%
Nissan	10%

Third Normal Form A relation is in 3NF when it is in 2NF and every non-key attribute depends on a primary key that is not transponent. Simply put, the second rule requires that all non- key fields whose contents can be referenced by multiple table records be converted to separate tables. Consider the table:

<u>Model</u>	Shop	Telephone
BMW	Real-auto	87-33-98
Audi	Real-auto	87-33-98
Nissan	Next-Auto	94-54-12

The table is in 2NF, but not in 3nf. In the relationship, the Model attribute is the primary key. Cars do not have personal phones, and the phone depends only on the store. Thus, the relationships have the following functional dependencies: Model → Store, Store → Phone, Model → Phone.

Test questions:

- 1) What commands are used to view available databases?
- 2) what is the difference between numeric and associative array?
- 3) what must be defined in two tables to join them?

Literature

1. Allen, G. Taylor SQL for dummies / Allen G. Taylor. - M.: Dialectics, Williams, 2015. - 416 p.
2. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
3. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 13.

Topic: Database objects (view, sequence, index, synonym).

subtopics: logical elements, components in hardware nodes

A database object is any specific object in a database that is used to store or reference data. Anything that we do from the create command is called a database object. It can be used to store and manipulate data. Some examples of database objects are: view, sequence, indexes, etc.

A table is the basic unit of storage; it consists of rows and columns. View - logically represents a set of data from one or more tables.

Sequence - generates primary key values. Index - improves the performance of some queries.

Synonym - alternative name of an object. Different forms of database:

Table - This database object is used to create a table in the database Syntax:

```
CREATE TABLE [schema.]table
(column datatype [DEFAULT expr][, ...]);
```

Example:

```
CREATE TABLE dept
(deptno NUMBER(2),
dname VARCHAR2(14),
loc VARCHAR2(13));
```

Conclusion:

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

DESCRIBE

dept;

Test questions:

- 1) What converter is used in the function printf?
- 2) What is the difference between a numeric array and an associative array?
- 3) How to create a database in a many to many relationship-to-many?"

Literature

1. Allen, G. Taylor SQL for dummies / Allen G. Taylor. - M.: Dialectics, Williams, 2015. - 416 p.
2. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
3. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

Lecture No. 14.

Topic: Division (Distribution), demand cost, demand optimization, indexation.

subtopics: tablespaces

Many SQL servers support horizontal table partitioning. What does this mean? Partitioning is dividing large tables into logical parts according to selected criteria. At the lowest level, these are physically different files, 3 for each section (table description, index file, data file). In the default configuration, they have different tablespaces (for example, InnoDB allows you to configure separate storage or even specific tables at the database level).

What it looks like:

```
CREATE TABLE orders_range ( customer_surname VARCHAR(30), store_id INT,
salesperson_id INT, order_date DATE, note VARCHAR(500)
) ENGINE = MYISAM
PARTITION BY RANGE( YEAR(order_date) ) ( PARTITION p_old VALUES LESS THAN(2008),
PARTITION p_2008 VALUES LESS THAN(2009), PARTITION p_2009 VALUES LESS
THAN(MAXVALUE)
);
```

So what do we get as a result? The first "table" stores data for the "archive" period, up to 2008, the second - up to 2009, and the third - everything else. The most important thing is that there is no need to rewrite / optimize queries: select * from order_range, where order_date = '2009-08-01';

Test questions:

- 1) Matin orisi men matin aimagy bir-birinen kalay erekshelededi?
- 2) Derekkordyn reserve koshirmesin zhasau ushin kanday kanday koldan kerek
- 3) Kop-kop katynasta malimette koryn qalay qyruqa boladas?

Literature

1. Allen, G. Taylor SQL for dummies / Allen G. Taylor. - M.: Dialectics, Williams, 2015. - 416 p.
2. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
3. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV- Petersburg, 2017. - 288 p.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

Lecture No. 15.

Topic: OLAP, DWH, ETL. Other relational DBMS Subtopics: DBMS types

Classifications of DBMS by the method of access to the database (databases): File-server. Data files are located centrally on the file server. The DBMS is located on the computer (workstation) of each client. Access to DBMS data is carried out via a local network. Examples: Microsoft Access, FoxPro, Visual FoxPro, Paradox, dBase. Client-server. The DBMS is located on the server together with the database and has direct access to the database. All client requests for data processing are processed centrally by the client-server MBR.

Embedded. A DBMS that can be delivered as part of a software product, without requiring its own installation procedure. An embedded DBMS is designed to store data locally for your application and is not intended for shared use over a network. A Data Warehouse is a domain-specific information database specifically designed for reporting and business analysis to support decision making in an organization. In a data warehouse, data is not created: i.e., it comes from external sources and is not corrected.

OLAP (Online Analytical Processing, Real-time Analytical Processing) is a data processing technology that consists of preparing summary information based on large data sets structured according to the multidimensional principle. An example with a telecom operator (providing a new tariff). The OLAP cube contains information about the master data and dimensions (aggregates). The cube contains all the information that may be needed to answer any question. In most units, a full calculation is often performed only for some dimensions, while the rest are performed "on demand". ETL (Extract, Transform, Load) is a complex process of transferring data from one application to another.

Extract, transform, load: obtaining data from external sources; transforming and refining it to meet the needs of the business model; and loading it into a data warehouse. In computing, a three-tier architecture, which is synonymous with three-tier or multitier architecture, takes the following application components: A client application connected to an application server (usually called a thin client or terminal), which in turn connects to a database server. The client is the previous (usually graphical) component, representing the first tier, the specific application for the end user. The first tier should not have direct contact with the database (for security requirements), should be loaded with basic business logic (for scalability requirements), and maintain the status of the application (for reliability requirements). This can be the simplest business logic, and can usually be obtained from the first tier: authorization interface, encryption algorithms, embedded values. The application server is located at the second tier. The second tier covers most of the business logic. Beyond this are the fragments exported to terminals (see above), as well as stored procedures and triggers, which are immersed in the third level. The database server provides data storage and switches to the third level. Typically, this is a standard relational or object-oriented database management system. If the third level is a database with stored procedures, triggers, and a schema describing the application in terms of the relational model, then the second level is created as a programming interface that connects client components with the database application logic.

Test questions:

- 1) What is the difference between a text box and a text area?
- 2) how to view table structure?
- 3) What are three benefits of using a system like Smarty?

Literature

1. Allen, G. Taylor SQL for dummies / Allen G. Taylor. - M.: Dialectics, Williams, 2015. - 416 p.
2. James, R. Groff SQL. The Complete Guide / James R. Groff, Paul N. Weinberg, Andrew J. Oppel. - M.: Williams, 2014. - 960 p.
3. Dunaev, V.V. Databases. SQL language for students / V.V. Dunaev. - M.: BHV-Petersburg, 2017. - 288 p.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	---	------------------

Lab #1. Introduction to SQL. Distributions. Installing Microsoft SQL Server.

Target: teach how to correctly configure Microsoft SQL Server 2008 on both the server and client sides.

Theoretical material: Before performing the lab, it is recommended that you study Lecture 1, “Introduction to Database Theory,” which covers the basic concepts of databases, and Lecture 2, “Microsoft SQL Server 2008 Components,” which covers in detail all the main components of MS SQL Server 2008 that will be configured during this lab.

Reporting requirements: during the lab work, you will need to write a report. The information needed for the report will be indicated in the work by the words "Write down in the report...".

Task 1. Explore the SQL Server Configuration utility.

Instructions for implementation:

1. On the server, run the SQL Server Configuration Manager utility and use it to determine the list of services running on the server. Record this list in a report.
2. Note: Additional options for managing the operation of services are provided by the Windows XP Services administration utility, which is accessed via the Control Panel – Administrative Tools folder.
3. On a server with installed MS SQL Server 2008, use the Services utility to determine the parameters for starting MS SQL Server services and record them in the report. (If you do not have access to the Services utility, then use SQL Server Configuration Manager).
4. Determine which network libraries can be used to establish a connection to MS SQL Server (see Fig. 1.1). Which libraries are currently active? Record this information in the report.

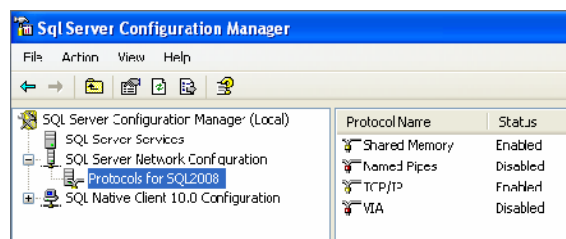


Fig. 1.1. Server-side protocols through which you can connect to it

5. Using SQL Server Configuration Manager, determine which network libraries the client can use to connect to MS SQL Server (see Fig. 1.2). Write the list of libraries to the report.

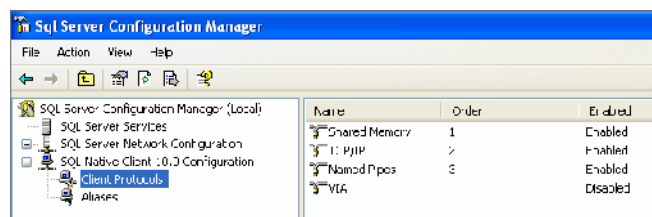


Fig. 1.2. Client-side protocols through which it can connect to the server

Task 2. Creating a server alias for client connections.

Before you start working with the server, you need to configure the connection parameters. To do this, in the SQL Server Configuration Manager utility, in the SQL Native Client Configuration – Client Protocols node, add the network library configuration, then specify the logical name of the server, select the required library from the list and define its parameters. For example, when interacting based on the TCP/IP stack, you need to specify the name of the network node on which MS SQL Server is installed and the TCP port number used for the connection (see Fig. 1.3).

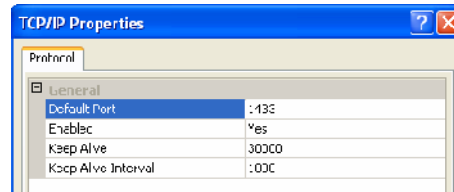


Fig. 1.3. TCP/IP protocol properties

Instructions for implementation:

1. Determine and record in the report the network name of the computer on which MS SQL Server is installed.
2. In the context menu (right mouse button) for the SQL Native Client Configuration – Aliases node, execute the New alias... command, set the Alias Name to MyServ, select the TCP/IP protocol, and specify the previously defined name in the Server field.
3. What port number should be specified when setting up this connection and why? Record in the report.

Task 3. Establish a connection to the SQL server.

Instructions for implementation:

1. On the workstation – the client computer – start SQL Server Management Studio and select from the list the logical server name MyServ, defined in the previous task using the SQL Server Configuration Manager utility. If the required server is not in the list, you can select <Browse for more...> and find the required server in the list of servers to which a connection can be made.
2. Connect to the MyServ server using MS SQL Server authentication: account "sa", no password.
3. To write a new query, you need to execute the New Query command located on the SQL Server Management Studio toolbar. This will open a new tab that provides the following options:
 - a header that specifies the logical name of the server, the current database, and the name of the user who established the connection;
 - query area used to enter queries passed to MS SQL Server;
 - the results area, which displays the results of the query, and the display method is set by the Messages buttons (in the formtext) and Results (in the form of a table), respectively.
4. Using the SELECT @@version command, determine and write to the report information about the version of MS SQL Server and operating system used (the query result should be displayed in text form) (Fig. 1.4).

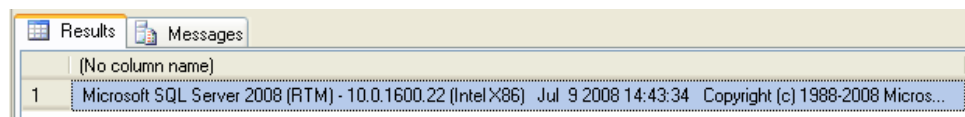


Fig. 1.4. Information about the MS SQL Server version

Note: To execute a query, you need to run the Query – Execute (F5) command, and to analyze the correctness of its syntax, you can use the Query – Parse (Ctrl+F5) command.

SQL Server Management Studio allows you to open multiple query windows and work with multiple databases simultaneously. Each window establishes its own connection to MS SQL Server based on different user accounts and their passwords. To create a new connection, use the File – New – Database Engine Query command.

The contents of the current connection request area can be saved to a file on an external drive using the File – Save command.

5. Use the Object Explorer panel to determine the names of the supported databases and which server databases are system databases (to do this, expand the Databases node in the Object Explorer panel). Record this information in the report.

Task 4. Study the configuration parameters of MS SQL Server.

Configuration of the MSSQLServer service can be performed either by a special stored procedure executed in the SQL Server Management Studio utility, or graphically by means of the same utility. The choice of method does not matter, since the graphic method provides access to system data using the same stored procedure, only in a more visual form.

Instructions for implementation:

1. To change the service settings using SQL Server Management Studio, select the desired server in Object Explorer and select Properties from the context menu. In the dialog box that appears, you can configure all the necessary settings.
2. Display the list of MyServ server parameters (see Fig. 1.5).

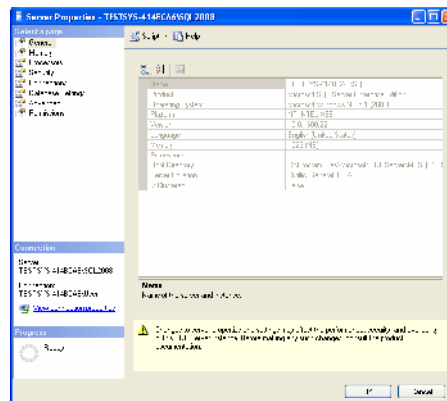


Fig. 1.5. MS SQL Server 2008 Properties

The General tab displays basic information about the system: operating system version, memory size, number of processors, etc., as well as server service startup parameters.

The Memory tab allows you to manage memory allocation for MS SQL Server actions: either dynamic memory management or set a fixed size.

The Security tab is used to define the user authentication type and also to define server access audit parameters. You can configure the server to use a specific account under which the MSSQLServer service will be launched.

The Connections tab allows you to configure client connections to the server. The maximum number of users that can simultaneously connect to the server. If zero is specified, their number is 32767.

The Advanced tab contains some general server settings. For example, you can define the default language for server messages or adjust year 2000 support, which determines how the last two digits of the year are interpreted.

The Database Settings tab is used to specify the settings for newly created databases: index parameters and parameters for working with backup devices, and the database recovery time.

3. Determine and record in the report the server's root directory, the number of processors in the system, the user authentication type, and the maximum number of users supported by the server.
4. Explore other MS SQL Server properties available in this dialog.

Microsoft SQL Server. Lab #2. Introduction to Transact-SQL

Objective: learn the basic operators and algorithmic constructs of Transact-SQL.

Theoretical material: Before performing the lab, it is recommended that you study Lecture 3, "General Information about Transact-SQL," since it provides a general description and purpose of the SQL language, and also covers all the algorithmic constructs available in SQL.

Reporting requirements: based on the results of the work, you need to present a set of SQL scripts that solve the problems from the "Independent Work" section.

Task 1. Connect to the MyServ server using the Management Studio utility.

Instructions for implementation:

1. Launch SQL Server Management Studio from the Start menu – Programs – Microsoft SQL Server 2008.
2. Select authentication type: SQL Server Authentication. Specify User name: sa, and Password: empty and click Connect.

Task 2. Determine whether the user has the right to select from the Product table.

Directions for implementation:

1. Set as current database AdventureWorks2008. This can be done in one of the following ways:
 - run the command USE AdventureWorks2008;
 - Select AdventureWorks2008 from the drop-down list (see Fig. 2.1).

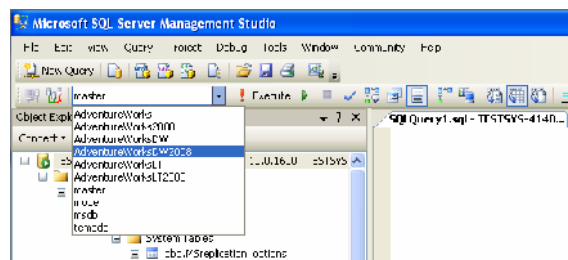


Fig. 2.1. Selecting the current database

2. Using the function `object_id` let's get the identifier of the Product table.
3. Function `Permissions` returns a 32-bit mask, where each bit represents the current user's right to a certain action. The value of the bit depends on the parameters with which the function was called. The `Permissions` function can be called in one of three ways:
 - without parameters – returns a list of permitted user actions that the user can perform in the current database;
 - with one parameter `object_id` – returns a list of valid actions on an object, for example, on a table;
 - with two parameters: `object_id` and `attribute` – returns the allowed actions on the attribute.

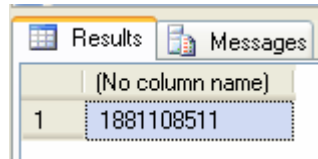
The table lists the permitted actions according to the different ways of calling the `Permissions` function. Each action is indicated by the number and decimal value of the corresponding bit.

Table 2.1. Permitted actions of the function call `Permissions`

Bit number	Bit (in decimal form)	Permitted action		
		No parameters	Object_id	Object_id and attribute
0	1	CREATE DATABASE	SELECT ALL	SELECT
1	2	CREATE TABLE	UPDATE ALL	UPDATE
2	4	CREATE PROCEDURE	REFERENCES ALL	REFERENCES
3	8	CREATE VIEW	INSERT	
4	16	CREATE RULE	DELETE	
5	32	CREATE DEFAULT	EXECUTE	
6	64	BACKUP DATABASE		
7	128	BACKUP LOG		
8	256	Reserved		
12	4096		SELECT ANY	
13	8192		UPDATE ANY	
14	16384		REFERENCES ANY	

In this case, we are interested in the least significant (zero) bit – the right to SELECT ALL.

1. Example of a function call `Permissions` (Fig. 2.2):
SELECT Permissions (object_id('production.product'))



(No column name)
1 1881108511

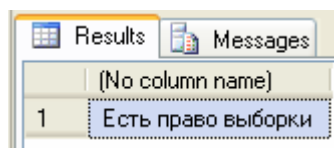
Fig. 2.2. Result of the Permissions function

- To determine whether the bit we are interested in is set, we can use the bitwise AND (&) operation, then the final result will look like this:

```

IF                                     Permissions
(object_id('production.product'))&1=1SELECT
'Have select permission'
ELSE
SELECT 'No selection rights'

```



(No column name)
1 Есть право выборки

Fig. 2.3. Result of the Permissions function for outputting a text description

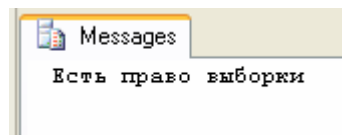
- Instead of the SELECT operator, you can use the PRINT operator, which simply allows you to output data as text, rather than as a table, as SELECT does. Then the code will look like this:

```

IF                                     Permissions
(object_id('production.product'))&1=1PRINT 'Have
the right to select'
ELSE
PRINT 'No selection rights'

```

And the result will be displayed not on the Results tab, but on the Messages tab (Fig. 2.4).



Messages
Есть право выборки

Fig. 2.4. Result of using PRINT

Task 3. Display the name of the computer on which the command is executed. If the computer name is more than nine letters, then display only the first six letters.

Directions for implementation:

- To get the computer name, use the function `HOST_NAME()`.
- To determine the length of a string, you can use the function `DataLength`.
- Example of completing the task:

```

IF (DATALENGTH(HOST_NAME()) / 2 > 9)
SELECT LEFT(HOST_Name(), 6) + '...'
ELSE
SELECT HOST_Name()

```

Comment. Note that we divided the result of the `DATALENGTH` function into two. This is because `HOST_NAME()` returns the `nvarchar` type, in which two bytes are allocated for each character.

Task 4. Count the number of digits in a number.

Directions for implementation:

- One from the possible solutions to this problem (see Fig. 2.5): DECLARE

```

@num int, @cnt int
SET @num = 19
IF (@num = 0) SET @cnt = 1
ELSE BEGIN
SET @cnt = 0

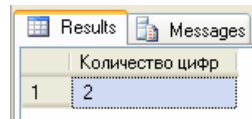
```



```

WHILE (@num <> 0) BEGIN
    SET @cnt = @cnt + 1
    SET @num = @num / 10
END
END
SELECT @cnt AS 'Number of digits'

```



Количество цифр
2

Fig. 2.5. Result of the code execution

Independent work.

1. Optimize the code for solving task 4: get rid of the conditional operator, only the loop should remain WHILE, and the code should still work for both zero and positive numbers, as well as negative ones.
2. Determine if a user has permission to access a field 'CardNumber', tables CreditCard in the database AdventureWorks2008. If access is available, then display "Access available", otherwise – "Access denied".
3. Write a program to convert weight from pounds to kilograms (1 pound equals 453.6 g). The result should be displayed as follows, for example:
3.3 lb(s) is 1 kg 496 g.
4. Display information about the current server, account, and database user names as follows:
5. You are logged on to server User400-01\SQL2008 as User400-01\User with dbo permissions.
6. Determine the number of hours and minutes that have passed since the MS SQL Server services were started.
7. Determine if the current year is a leap year.
8. Find the sum of numbers in a given string of characters.
9. Determine the amount of payment for sending a telegram. A period is used to indicate the end of a telegram. The cost of one word is 33 kopecks, output the result indicating the number of rubles and kopecks. The line may contain any number of spaces.
10. Using the Caesar cipher, encrypt a given string of text. The idea behind this encryption method is that the alphabet is arranged clockwise. To encrypt, the letters of the text are replaced by letters that are a given number of letters (shift) clockwise.
11. The user's last name, first name and patronymic are known. Find his/her personality number. The rule for obtaining the personality number: each letter is assigned a number - the ordinal number of the letter in the alphabet. These numbers are added up, if the resulting sum is not a single-digit number, then the digits of the number are added up again and so on until a single-digit number is obtained.

Microsoft SQL Server. Lab #3. Data Sampling

Target: teach how to use the SELECT statement and its main sections to select data according to specified conditions.

Theoretical material: Before performing the lab work, it is recommended to study Lecture No. 4 "Data Selection", which studies one of the main SQL language operators – SELECT and explains the purpose of each of its sections.

Reporting requirements: based on the results of the work, present a set of SQL scripts that solve the problems from the "Independent Work" section.

Task 1. Connect to the MyServ server using the Management Studio utility.

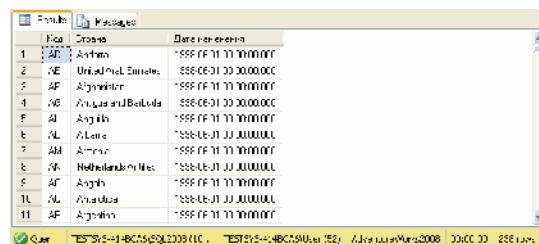
Instructions for implementation:

1. Launch SQL Server Management Studio from the Start menu – Programs – Microsoft SQL Server 2008.
2. Select authentication type: SQL Server Authentication. Specify User name: sa, and Password: empty and click Connect.

Task 2. Get a list of countries with their code and the last date of record modification from the CountryRegion table of the AdventureWorks2008 database.

Directions for implementation:

1. To complete the task we will need attributes `CountryRegionCode`, `[Name]`, `ModifiedDate` from the table `Person.CountryRegion`. Note that the table name is preceded by the Person schema name, which is used to manage objects related to employees and departments.
2. To rename columns, we will use the keyword `AS`.
3. Make sure, what `BDAdventureWorks2008` is current, and execute the following query: `SELECT CountryRegionCode AS 'Code', [Name] AS 'Country', ModifiedDate AS 'Modified Date' FROM Person.CountryRegion`
The result of executing the query is shown in Fig. 3.1.



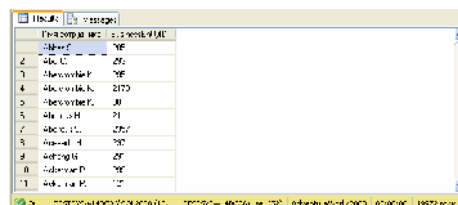
Code	Country	Date Modified
AF	Afghanistan	2008-06-21 00:00:00
AE	United Arab Emirates	2008-06-21 00:00:00
DZ	Algeria	2008-06-21 00:00:00
AZ	Azerbaijan	2008-06-21 00:00:00
BD	Bangladesh	2008-06-21 00:00:00
BH	Bahrain	2008-06-21 00:00:00
KB	Cambodia	2008-06-21 00:00:00
CN	China	2008-06-21 00:00:00
CO	Colombia	2008-06-21 00:00:00
CR	Costa Rica	2008-06-21 00:00:00
CZ	Czechia	2008-06-21 00:00:00

Fig. 3.1. List of countries

Task 3. Get a list of employees with their last name and first character of their first name, as well as their identification number. The list should be sorted by the employee's last name and first name.

Directions for implementation:

1. We will have to rename the first column using the operator `AS`, since it requires combining data from two columns: `LastNameAndFirstName`.
2. To get only the first letter of the name, we will use the Substring function.
3. To sort the resulting table, you need to use the section **ORDER BY**.
4. Run the following code:
`SELECT LastName+' '+Substring(FirstName,1,1)+' ' as [Employee Name], BusinessEntityID FROM Person.Person ORDER BY [Employee name]`
The result of executing the query is shown in Fig. 3.2.



Employee Name	BusinessEntityID
Aftab Khan	281
Aftab Khan	282
Aftab Khan	283
Aftab Khan	284
Aftab Khan	285
Aftab Khan	286
Aftab Khan	287
Aftab Khan	288
Aftab Khan	289
Aftab Khan	290
Aftab Khan	291

Fig. 3.2. List of employees

Comment. By default, sorting is performed in ascending order, which corresponds to the reserved word `ASC` (may not be specified); for sorting in descending order, `DESC` is specified.

Task 4. Get a list of products whose price is in the range from \$12 to \$20, sorted by price.

Directions for implementation:

1. To select records according to a given criterion, you must use the section **WHERE**.
2. You can use a compound condition, then the code will look like this:
`SELECT [Name], ListPrice From Production.Product WHERE (ListPrice>=12) and (ListPrice<=20)`

ORDER BY 2

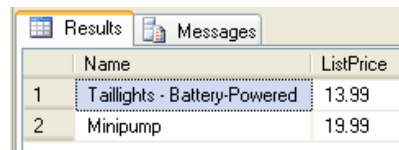
- Another option: replace the two comparison operators with one logical operator **BETWEEN**, with the help of which you can get an answer to the question of whether the value lies in the specified range:

```
SELECT [Name], ListPrice From Production.Product
```

```
WHERE ListPrice Between 12 and 20
```

```
ORDER BY ListPrice
```

The result of executing the query is shown in Fig. 3.3.



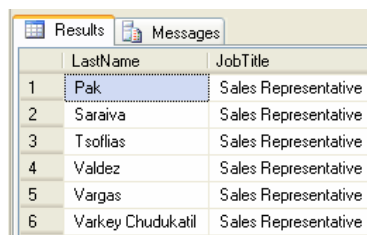
	Name	ListPrice
1	Taillights - Battery-Powered	13.99
2	Minipump	19.99

Fig. 3.3. List of products with price from \$12 to \$20

Task 5. Display the names of employees and their positions living outside USA.

Directions for implementation:

1. To search for a pattern of character strings, a logical operator is used. **LIKE**, which is most often used in situations where an exact match is not known.
2. In the template we will need to specify a service character **%**, which means any string consisting of 0 or more characters.
3. Using table *HumanResources.vEmployee* Let's create a query: Select LastName, JobTitle From HumanResources.vEmployee Where CountryRegionName Not Like '%United States%' The result of executing the query is shown in Fig. 3.4.



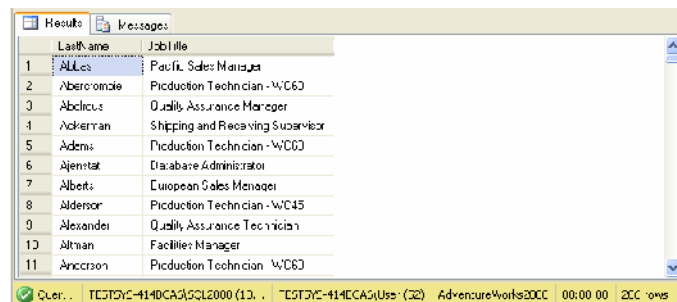
	LastName	JobTitle
1	Pak	Sales Representative
2	Saraiva	Sales Representative
3	Tsoflias	Sales Representative
4	Valdez	Sales Representative
5	Vargas	Sales Representative
6	Varkey Chudukatil	Sales Representative

Fig. 3.4. Employees residing outside the United States

Task 6. Get a list of employees from the USA and Canada.

Directions for implementation:

1. To determine whether an expression matches one of the values listed in a given list, a logical operator is used. **IN**. This operator can always be written as a group of conditions united by an operand **OR**.
2. Request using the operator **IN** will look like this: Select LastName, JobTitle From HumanResources.vEmployee WHERE CountryRegionName IN ('United States', 'Canada') The result of the query is shown in Fig. 3.5.



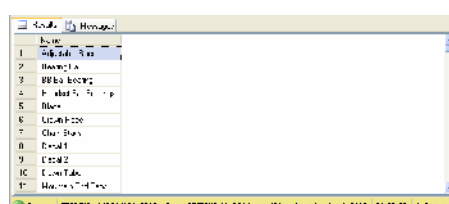
	LastName	JobTitle
1	Alles	Paulk, Sales Manager
2	Abercrombie	Production Technician - WC60
3	Abdricus	Quality Assurance Manager
4	Ackerman	Shipping and Receiving Supervisor
5	Adams	Production Technician - WC60
6	Ajennat	Database Administrator
7	Alberts	European Sales Manager
8	Alderson	Production Technician - WC45
9	Alexander	Quality Assurance Technician
10	Altman	Facilities Manager
11	Amerson	Production Technician - WC60

Fig. 3.5. Employees from the USA and Canada

Task 7. Get a list of products that do not have a color specified.

Directions for implementation:

1. To check for the absence of a value, the IS NULL sampling function is used.
2. Let's make it up next query on table *Product*:
SELECT [Name] FROM Production.Product
WHERE Color IS NULL
The result of executing the query is shown in Fig. 3.6.



	Name
1	Black
2	Black
3	Black
4	Black
5	Black
6	Black
7	Black
8	Black
9	Black
10	Black
11	Black

Fig. 3.6. Products for which color is not specified

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

Independent work

1. Display information about the delivery methods of the product (*ShipMethod*), if the active user has the right to receive this data.
2. Construct a query that returns a list of employees (table*Employee*) indicating them*EmployeeID*, date of birth (in dd.mm.yyyy format) and age as of today.
3. Hint: To convert a value of one type to a value of another type, use the function: **CONVERT(type, expression[, style])**.
4. Identify the list of products that have been discontinued (using*SellEndDate*), indicating their color.
5. Get warehouse locations from*Production.Location*, containing 'Paint' in their name.
6. Display a list of countries where customers are registered. The selection should be made from the view*IndividualCustomer*, and not from the table, because the table does not contain enough information for such a request.

Microsoft SQL Server. Lab #4. Stored Procedures

Target: teach how to create and call custom stored procedures, as well as pass parameters to them.

Theoretical material: Before performing the lab, it is recommended that you study Lecture 3, “General Information about Transact-SQL,” which provides the basics of SQL language syntax, and Lecture 5, “Auxiliary Database Objects,” which provides basic information about stored procedures.

Reporting requirements: based on the results of the work, present a set of SQL scripts that solve the problems from the “Independent Work” section.

Task 1. Create a stored procedure that outputs the name of the computer on which the command is executed. If the computer name is more than nine letters, then output only the first six letters.

Instructions for implementation:

1. To get the computer name, use the function **HOST_NAME()**.
2. For To create a procedure, run the following code:

```
CREATE PROCEDURE Test AS
IF (DATALENGTH(HOST_NAME()) / 2 > 9)
SELECT LEFT(HOST_Name(), 6) + '...'
ELSE
SELECT HOST_Name()
```

Comment. The stored procedure will be created in the current database.

3. Make sure that the stored procedure has been created (see Figure 4.1).

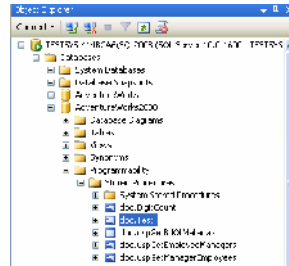


Fig. 4.1. List of stored procedures in the database

4. Call the created stored procedure by executing the following code: TEST

Task 2. Create a stored procedure that takes a number as a divisor and returns the number of digits in it through the OUTPUT parameter.

Instructions for implementation:

1. Create a stored procedure DigitCount with two parameters:

```
CREATE PROCEDURE DigitCount
```

```
    @num int,
```

```
    @cnt int OUTPUT
```

```
AS
```

```
IF (@num = 0) SET @cnt = 1
```

```
ELSE BEGIN
```

```
    SET @cnt = 0
```

```
    WHILE (@num <> 0) BEGIN
```

```
        SET @cnt = @cnt + 1
```

```
        SET @num = @num / 10
```

```
    END
```

```
END
```

2. Call procedureDigitCountand make sure it works correctly: DECLARE

```
@num int = 1249
```

```
DECLARE @cnt int
```

```
EXEC DigitCount @num, @cnt OUTPUT
```

```
SELECT @cnt
```

*Comment.*Please note that in this case the EXEC keyword is required before calling the stored procedure, because the stored procedure is specified in a block of other SQL commands, and is not executed alone.

Independent work

1. Determine the number of system stored procedures.

Indication. Design the solution to the following problems in the form of stored procedures.

2. Determine the season for a given date: fall, winter, spring or summer.

*Comment.*To control the date format, use the following command: SET DATEFORMAT Format.

The Format option specifies the date input format and can take the following values –

mdy, dmy, ymd, myd and dym (where m, d and y are day, month and year respectively).

3. The user's date of birth is known. Check whether he or she is 16 years old or younger.
4. Determine whether a character string is an email address. Make the source code of the procedure unreadable. Prove that the steps are correct. Hint: the string must contain the character@, after which there must be at least one dot.
5. Delete all procedures you have created.

Microsoft SQL Server. Lab #5. Microsoft SQL Security System Server

Target:teach how to use system stored procedures to manage MS SQL Server logins and database users, as well as to allow and prohibit certain actions to be performed by a certain user.

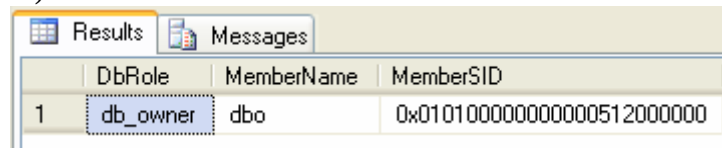
- Log in again under the sa account, since further actions will again require administrator rights.
- To assign a login account to a built-in server role, use the following procedure:
sp_addsrvrolemember 'TempUser', 'securityadmin'

Task 4. Identify the list of AdventureWorks2008 database roles and role members.

db_owner.

Instructions for implementation:

- Execute sp_helprole to list both built-in and user-defined database roles.
- Use the sp_helprolemember 'db_owner' command to determine the members of the db_owner role (see Figure 5.3).



	DbRole	MemberName	MemberSID
1	db_owner	dbo	0x010100000000000512000000

Fig. 5.3. List of members of the db_owner role

Task 5. Create a new database user for the TempUser login.

Instructions for implementation:

- Add a user using a stored procedure: sp_adduser 'TempUser', 'MyFirstUser'
- Use sp_helpuser to verify that the user was added. What role was assigned to the user?
- Add the db_datareader role to the user:
sp_addrolemember 'db_datareader', 'MyFirstUser'

Task 6. Set up access rights for the user Andy: explicitly grant the right only to select from the Product table and update only the Name and ListPrice fields of this table.

Instructions for implementation:

- The following command grants the TestUser user in the AdventureWorks2008 database the right to select and modify data in the Orders table in that database: GRANT select, update on AdventureWorks2008.Production.WorkOrder to TestUser
- The following command grants user Andy permission to only select data from the Name and ListPrice fields of the Product table in the AdventureWorks2008 database: GRANT select on AdventureWorks2008.Production.Product (Name, ListPrice) to Andy

Task 7. Learn how to perform the above functions using the graphical interface of the Management Studio utility.

Instructions for implementation:

- You can view the list of existing accounts and their parameters by selecting the Logins group in the Security folder of the server.
- To create a new login account, you need to execute the New Login... command from the context menu of the Logins node, and in the dialog box that appears, specify:
 - General tab: user name, authentication type (if authenticated by means of MS SQL Server – set a password), database to which the user connects automatically, default language;
 - Server Roles tab: server roles that the created account will belong to;
 - User Mapping tab: access to one of the databases created on the server, in the User field enter the name of the database user and include the created user in one of the existing roles.

Comment. To change the parameters of an existing user account, select it from the list and execute the context menu command Properties; to delete it, select Delete.

- To display the list of server roles, select the Server Roles group in the server's Security folder. Viewing the users included in this role and the permissions assigned to it is done by executing the Properties context menu command.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

- Note: Built-in server roles cannot be removed from the system, and their defined permissions cannot be changed. It is also prohibited to create your own server roles.
- The Security/Users group of a database is used to view and manage user parameters for a particular database. Accounts are displayed in the User Name field, and the corresponding login accounts are displayed in the Login Name field.

To create a new database user, you must run the New User... command, then enter the user name in the User name field, and select the appropriate login account from the Login Name list. You can also include the user in database roles.

Comment. To change account parameters, use the Properties command, and to delete them, use the Delete command.

- The Roles group is used to display a list of database roles. To view users in this group, you must run the Properties command.
- To assign permissions to a security object, you must select it in the Users group (to change permissions for a specific database user) or in the Roles group (for permissions for a specific role). The Securables tab is used for these purposes.

The tab that appears lists all database objects with possible access rights. You can set one of three access states: grant (check mark), deny (cross) and implicit rejection (empty field) – in the corresponding field.

Task 8: Unassign an account role and delete the account

TempUser.

Instructions for implementation:

- Removing a role assigned to a user can be done using the procedure: `sp_droprolemember 'db_datareader', 'MyFirstUser'`
- To delete a database user, use the following procedures:
`sp_dropuser 'MyFirstUser'`
- Unassigning an account from a specific role is done using a stored procedure:
`sp_dropsrvrolemember 'TempUser', 'securityadmin'`
- To drop an account, execute the stored procedure: `sp_droplogin 'TempUser'`

Independent work

- List all server roles and the actions allowed to users of the dbcreator role.
- Determine which server role is assigned to the sa system account.
- Determine which role a user has the ability to create and delete login accounts.
- Changing the user account login password is performed using the `sp_password` procedure.
- Create your own login account with a connection to the AdventureWorks2008 database, prove the correctness of the actions performed. Assign the rights to create and change databases to the created account, prove the correctness of the actions performed. Connect to MS SQL Server using the created account, and create another user account for login, explain the result.
- Create a user manager for the AdventureWorks2008 database based on the login account you created earlier. Prove that you have done this correctly.
- Assign the manager user a role that only allows viewing the contents of the AdventureWorks2008 database.

Comment. To check the correctness of the actions performed, you can execute an arbitrary query to this database, for example, displaying the contents of the Production.WorkOrder table:

`SELECT * FROM Production.WorkOrder.`

- Forbid the user manager to view the AdventureWorks2008 database data by assigning the necessary role. How can I prove that the changes were made correctly?

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

9. How many database users can be created based on one user login account? Justify your answer.
10. Using SQL Server Management Studio, create a user account that uses SQL authentication to connect to the AdventureWorks2008 database by default and has diskadmin server role rights.
11. Define the list of users included in the diskadmin role and its permissions.
12. In the AdventureWorks2008 database, create a user based on the login account you created earlier.
13. For the previously created AdventureWorks2008 database user, determine what role this user is a member of and what is its purpose. Does this user have the right to select data from the Production.Product table of this database? Justify your answer and verify it by retrieving data using the SELECT * from Production.Product command.
14. In the AdventureWorks2008 database, create the managers role. For this role, define the right to select data from the Production.Product table of the AdventureWorks2008 database. Assign the managers role to the user created earlier. Does this user now have the right to select data? Check the conclusion. To what other objects of the AdventureWorks2008 database does this user have the right to access? Justify and check the conclusion.

Microsoft SQL Server. Lab #6. Database Management Objective: teach how to use

SQL commands to create and modify databases structures, as well as perform backups and restore databases from backups.

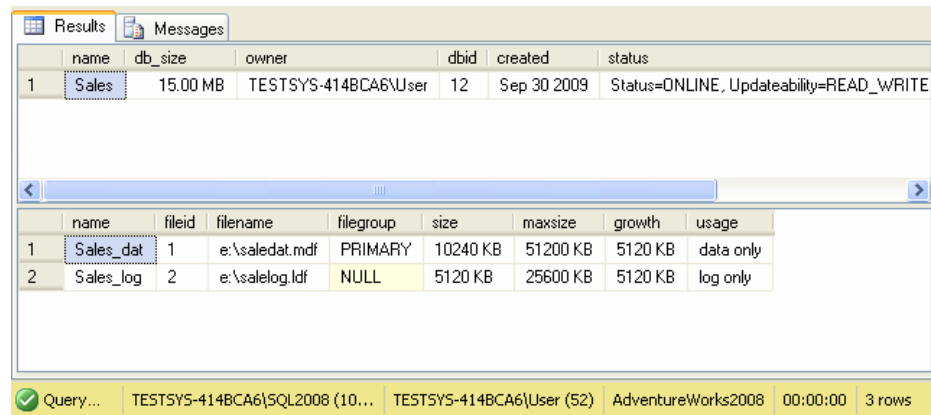
Theoretical material: Before performing the laboratory work, it is recommended to study lecture No. 7 “Database structure in MS SQL Server”, which examines the organization of the database within the MS SQL Server 2008 DBMS, as well as the main structural elements of the database.

Reporting requirements: based on the results of the work, submit a report with screenshots containing SQL commands and the results of their execution for each task from the section "Independent work".

Task 1. Create a Sales database with a log.

Directions for implementation:

1. Let's take advantage CREATE DATABASE function: CREATE DATABASE Sales
ON
(NAME = Sales_dat,
FILENAME = 'e:\saledat.mdf',
SIZE = 10,
MAXSIZE = 50,
FILEGROWTH = 5)
LOG ON
(NAME = Sales_log,
FILENAME = 'e:\salelog.ldf',
SIZE = 5MB,
MAXSIZE = 25MB,
FILEGROWTH = 5MB)
2. Run the following code to verify that the database was created with the required parameters (Figure 6.1):
sp_helpdbSales
The result of execution is shown in Fig. 6.1.



name	db_size	owner	dbid	created	status
1 Sales	15.00 MB	TESTSYS-414BCA6\User	12	Sep 30 2009	Status=ONLINE, Updateability=READ_WRITE

name	fileid	filename	filegroup	size	maxsize	growth	usage
1 Sales_dat	1	e:\saledat.mdf	PRIMARY	10240 KB	51200 KB	5120 KB	data only
2 Sales_log	2	e:\salelog.ldf	NULL	5120 KB	25600 KB	5120 KB	log only

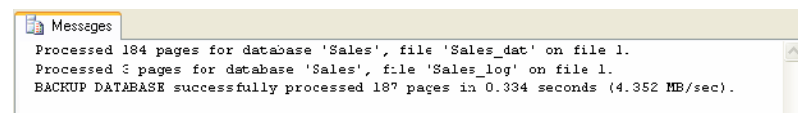
Query... TESTSYS-414BCA6\SQL2008 (10... TESTSYS-414BCA6\User (52) AdventureWorks2008 00:00:00 3 rows

Fig. 6.1. Database information

Task 2. Create a backup copy of the Sales database.

Directions for implementation:

- Let's create a backup device: sp_addumpdevice 'disk', 'backupdisk', 'e:\sales.bak'
 - Let's take advantage BACKUP function to perform backup: BACKUP DATABASE Sales TO backupdisk
- The result of execution is shown in Fig. 6.2.



Messages
Processed 184 pages for database 'Sales', file 'Sales_dat' on file 1.
Processed 3 pages for database 'Sales', file 'Sales_log' on file 1.
BACKUP DATABASE successfully processed 187 pages in 0.334 seconds (4.352 MB/sec).

Fig. 6.2. Result of executing the BACKUP operator

Independent work

Indication To prove that actions were performed correctly, you can use sp_helpdb procedures before and after performing some operation.

Task 1.

- Create a user with permission to create and delete databases.
- Create your own database with a main file size of 5 MB and a log file size of 2 MB. The main file can grow to 10 MB in 20% increments. The log file size can grow to 5 MB in 1 MB increments. The location of the files on the disk is D:\MSSQL\DATA.
- Make sure the database is created properly.

Task 2.

- Review the list of database options that can be set.
- Review the list of installed parameters of the database you created.
- Specify that the database is used by the owner only and in single user support mode.
- Make sure you change the database settings.

Task 3.

- Rename the database you created.

Task 4.

- Create your own file group.
- For your database, define another 2 MB file and place it in the created workgroup.
- Prove that your actions were correct.

Task 5.

- Set up automatic data compression for your database.

Task 6.

- Determine the disk space information consumed by the AdventureWorks2008 database.
- Shrink the AdventureWorks2008 database so that it contains only 25% of the space currently available to it.

3. Prove the correctness of the action performed.

Task 7.

1. For the AdventureWorks2008 database, add another file to the PRIMARY filegroup.
2. Compact the AdventureWorks2008 database. Justify your selections.
3. Prove that the database compression actually happened.

Task 8.

1. Create a backup copy of the AdventureWorks2008 database. Backup device physical name: D:\MSSQL\BACKUP\AdventureWorks_backup.bak.
2. Back up the AdventureWorks2008 database transaction log.

Task 9.

1. Remove the AdventureWorks2008 database.
2. Restore the deleted database from the device with the physical name AdventureWorks_backup.bak.
3. Prove the correctness of the action performed.

Microsoft SQL Server. Lab #7. Data selection from several tables

Target: teach how to use the JOIN construct to outer and inner join two or more tables.

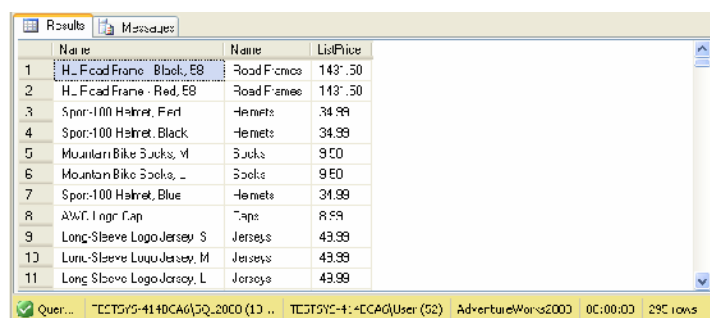
Theoretical material: Before performing the lab work, it is recommended to study Lecture No. 4 “Data Selection”, which studies one of the main operators of the SQL language – SELECT, and explains the purpose of each of its sections, and also considers issues of joining several tables.

Reporting requirements: based on the results of the work, present a set of SQL scripts that solve the problems from the “Independent Work” section.

Task 1. Get a list of products with their categories.

Instructions for implementation:

1. Since the Product table of the normalized AdventureWorks2008 database stores only the product category ID, to obtain a list of products with their categories, you need to join the Product and ProductSubcategory tables.
2. Before specifying the field in the SELECT statement, you will need to use the Product and ProductSubcategory table names to resolve conflicts, since both tables have Name fields.
SELECT Product.Name, ProductSubcategory.Name, Product.ListPrice
FROM Production.Product INNER JOIN Production.ProductSubcategory
ON Product.ProductSubcategoryID = ProductSubcategory.ProductSubcategoryID
The result of the query is shown in Fig. 7.1.



	Name	Name	ListPrice
1	H_ Road Frame - Black, E8	Road Frames	1431.50
2	H_ Road Frame - Red, E8	Road Frames	1431.50
3	Spor-100 Helmet, Red	Helmets	34.99
4	Spor-100 Helmet, Black	Helmets	34.99
5	Mountain Bike Socks, M	Socks	9.50
6	Mountain Bike Socks, L	Socks	9.50
7	Spor-100 Helmet, Blue	Helmets	34.99
8	AWF, Ingr Cap	Caps	8.99
9	Long-Sleeve Logo Jersey, S	Jerseys	43.99
10	Long-Sleeve Logo Jersey, M	Jerseys	43.99
11	Long-Sleeve Logo Jersey, L	Jerseys	43.99

Fig. 7.1. The result of joining two tables – products with a category indication

Comment. With this join, products that do not have a category specified are not included in the result set.

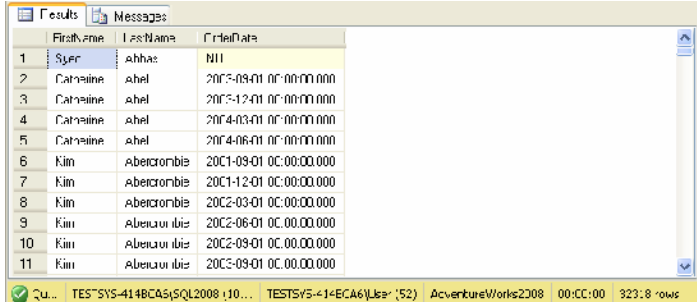
Task 2. Create a query that returns the customer's name and the date the order was placed.

Instructions for implementation:

- Run the following code:

```
SELECT FirstName, LastName, OrderDate FROM Person.Person
LEFT JOIN Sales.Customer
ON Customer.PersonID = Person.BusinessEntityID
LEFT JOIN Sales.SalesOrderHeader
ON Customer.CustomerID = SalesOrderHeader.CustomerID
```

The result of the query execution is shown in Figure 7.2.



	FirstName	LastName	OrderDate
1	Syer	Ahhaz	NULL
2	Catherine	Ahel	2013-09-01 00:00:00.000
3	Catherine	Ahel	2013-12-01 00:00:00.000
4	Catherine	Ahel	2014-03-01 00:00:00.000
5	Catherine	Ahel	2014-06-01 00:00:00.000
6	Kim	Abercrombie	2011-09-01 00:00:00.000
7	Kim	Abercrombie	2011-12-01 00:00:00.000
8	Kim	Abercrombie	2012-03-01 00:00:00.000
9	Kim	Abercrombie	2012-06-01 00:00:00.000
10	Kim	Abercrombie	2012-09-01 00:00:00.000
11	Kim	Abercrombie	2013-09-01 00:00:00.000

Fig. 7.2. The result of joining three tables - customers and the dates they placed orders For

people who did not place orders, the OrderDate field contains a NULL value.

With a right join (keyword RIGHT JOIN), all records from the right table are included in the result, regardless of whether there is a corresponding row for them in the left table or not. Modify the above query so that it produces the same results when using a right join.

Task 3. Using a cross join, you can list all possible delivery methods for goods in the Northwind database (supplied with MS SQL Server before AdventureWorks2008).

Instructions for implementation:

```
SELECT DISTINCT Suppliers.Country, Orders.ShipCountry
FROM Suppliers CROSS JOIN Orders
```

Independent work

- Display the last name and first name of customers from the vIndividualCustomer view, and the number (SalesOrderNumber) and date of their orders, starting from January 1, 2003, from the SalesOrderHeader table. If a company places multiple orders on the same day, only one record should be returned.

Indication: To limit the number of records in the resulting data set, you must specify a condition in the WHERE clause.

- Get a list of products with order number and customer, ordered on a certain day, for example, January 1, 1998. Design the solution as a stored procedure.
- Construct a query that returns the order number, the customer's first and last name, and the last name of the employee who processed the order.
- Create an order query for each product, specifying its name, price, and quantity using inner and outer joins.
- What is the difference between using inner and outer joins when solving problem #4?
- Suggest a cross join option for the AdventureWorks2008 database and implement it in SQL.
- Obtain a data set containing the following information: names and surnames of customers, their addresses; names of manufacturing companies, with their addresses; names and surnames of employees.

Indication: the last name and first name must be displayed as one field. Data selections should be made not from tables, but from the following views: vEmployee, vIndividualCustomer, vVendor.

Microsoft SQL Server. Lab #8. Analytical Data Sampling

Target: teach how to use aggregate functions and the GROUP BY section of the SELECT statement for analytical data retrieval.

Theoretical material: Before performing the lab, it is recommended that you study Lecture #4, “Data Selection,” which examines the SELECT operator and explains the purpose of each of its sections, as well as discusses issues of data grouping.

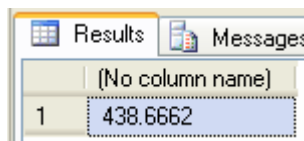
Reporting requirements: based on the results of the work, present a set of SQL scripts that solve the problems from the “Independent Work” section.

Task 1. Calculate the average price of goods.

Directions for implementation:

1. To calculate the average price, you need to use the aggregate function AVG.
2. The aggregate function will combine the values of the ListPrice field so that the following code will return only one number (Figure 8.1):

```
SELECT AVG(ListPrice) FROM Production.Product
```



Results	
	(No column name)
1	438.6662

Fig. 8.1. Average price of goods

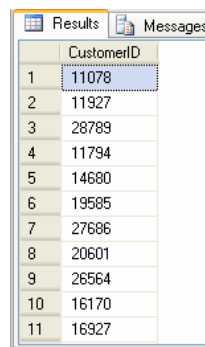
Task 2. Determine the names of customers who placed orders in the last accounting period.

Directions for implementation:

1. To make things simple, let's first get a list of only CustomerIDs, not customer names. You can use the following query to do this:

```
Select CustomerID From Sales.SalesOrderHeader
```

Where OrderDate=(SELECT Max(OrderDate) From Sales. SalesOrderHeader) The result of the query is shown in Fig. 8.2.



	CustomerID
1	11078
2	11927
3	28789
4	11794
5	14680
6	19585
7	27686
8	20601
9	26564
10	16170
11	16927

Fig. 8.2. IDs of customers who placed orders on the last accounting day

Comment. This query is executed in two stages: in the first stage, the subquery is executed as an independent query, which returns a value used in the second stage when executing the outer query.

2. To fully solve the problem, we will select the names of all customers from the Person table that are present in the list obtained in the previous paragraph. However, the Person table does not contain the CustomerID field, so we will also have to join it with the Customer table. The resulting code will look like this:

```
SELECT FirstName + ' ' + LastName AS
```

```
'CustomerName'FROM Person.Person
```

```
INNER JOIN Sales.Customer ON Customer.PersonID = Person.BusinessEntityID
```

```
WHERE CustomerID in
```

```
(SELECT CustomerID FROM Sales.SalesOrderHeader
```

```
WHERE OrderDate=(SELECT Max(OrderDate) FROM Sales. SalesOrderHeader)) The
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	---	---	------------------

result of the query is shown in Fig. 8.3.



	CustomerName
1	Alisha Raji
2	Chloe Harris
3	Kristi Fernandez
4	Tamara Lu
5	Devin Phillips
6	Rebekah Gonzalez
7	Caleb Lal
8	Marvin Munoz
9	Jacqueline Murphy
10	Chloe Martin
11	Christina Morris

Fig. 8.3. Names of customers who placed orders on the last accounting day

Comment. Such a query can also be designed as a table join. Such queries are processed much faster, so if the query does not need to be executed in several stages, the use of subqueries is not necessary.

Task 3. Determine the list of customers who ordered more than 300 units of goods at one time.

Directions for implementation:

1. For each order we need to get the total quantity of the product, for this we will need to create a related subquery.
2. In order not to write the full table name before the field names each time, it is convenient to use aliases. In this case, we will define the following aliases: for the SalesOrderHeader table, the alias is oh, and for SalesOrderDetail, the alias is od.
3. Resulting the code will look like this: `SELECT SalesOrderID, CustomerID
FROM Sales.SalesOrderHeader oh
WHERE 300<(SELECT sum(OrderQty)
FROM Sales.SalesOrderDetail od WHERE od.SalesOrderID=oh.SalesOrderID)`

Task 4. For each product, display information about its largest order: quantity of ordered product and order number.

Directions for implementation:

1. To complete the task, you need to use an inner join between the Product and SalesOrderDetail tables, as well as a related subquery that returns the maximum order for each product.
2. In the query, we will need to work with the SalesOrderDetail table both in the outer query and in the inner related subquery. Therefore, for this table, we will create two aliases: od1 and od2.
3. Resulting the code will look like this: `SELECT Name,
OrderQty, SalesOrderID
FROM Production.Product p INNER JOIN Sales.SalesOrderDetail od1
ON od1.ProductID=p.ProductID
WHERE OrderQty = (SELECT Max(OrderQty)
FROM Sales.SalesOrderDetail od2
WHERE od1.ProductID=od2.ProductID)`

Task 5. Determine the number of different products in each order.

Directions for implementation:

1. It is necessary to group records with the same SalesOrderID and count the number of records in each group by the ProductID field. For this, you can use the following query:
`SELECT SalesOrderID, count(ProductID) AS [Quantity] FROM
Sales.SalesOrderDetail
GROUP BY SalesOrderID`

Comment. When using GROUP BY, only one record is returned in the resulting data set for each defined group of values.

2. If for each order it is also necessary to calculate the total number of goods in the order, then the request should be supplemented with:

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
--	--	--	------------------

```
SELECT SalesOrderID,
count(ProductID) AS [Quantity],
sum(OrderQty) as [Total Quantity]
FROM Sales.SalesOrderDetail
GROUP BY SalesOrderID
```

Comment. When grouping records, the WHERE clause can also be used, in which case records that satisfy this condition are grouped.

Independent work

1. Determine the number of employees of a company in the AdventureWorks2008 database.
2. Calculate the maximum cost of delivery of orders.
3. Display a list of products whose price is higher than the average price.
4. Solve this problem using table join.
5. For each product, display information about its largest order: name, quantity and order number.
6. Note: Use grouping by product name and order number.
7. Display a list of products for which more than 50 orders have been received.

Microsoft SQL Server Lab 9: Using Views Objective: teach how to create and use views

when selecting data.

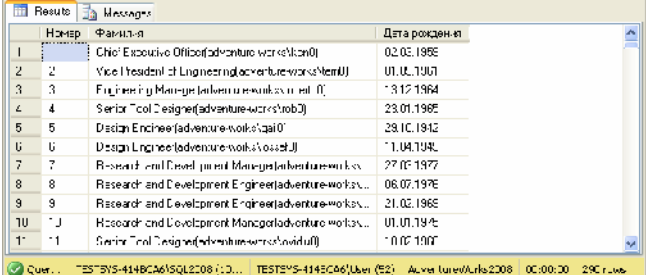
Theoretical material: Before performing the lab, it is recommended that you study Lecture 4, “Data Selection,” which examines one of the main SELECT operators, explains the purpose of each of its sections, and also examines issues of data grouping, and Lecture 5, “Auxiliary Database Objects,” which provides basic information about views.

Reporting requirements: based on the results of the work, present a set of SQL scripts that solve the problems from the “Independent Work” section.

Task 1. Create a view containing only such information about the employees of the AdventureWorks2008 company as: employee ID, his position and login (in one field), date of birth.

Directions for implementation:

1. Create a view:
CREATE VIEW InfoEmployees ([Number], [Last Name], [Date of Birth]) AS
SELECT BusinessEntityID, JobTitle + '(' + LoginID + ')', CONVERT (char(10), BirthDate, 104)
FROM HumanResources.Employee
2. To view the contents of a projection, the following query is executed (the result is shown in Figure 9.1):
SELECT * FROM InfoEmployees



Number	Фамилия	Дата рождения
1	Chief Executive Officer(adventure-works\adull)	02.03.1955
2	Vice President of Engineering(adventure-works\adull)	01.02.1901
3	Engineering Manager(adventure-works\adull)	13.12.1964
4	Senior Tool Designer(adventure-works\adull)	23.01.1965
5	Design Engineer(adventure-works\adull)	28.10.1942
6	Design Engineer(adventure-works\adull)	11.04.1944
7	Research and Development Manager(adventure-works\adull)	27.05.1977
8	Research and Development Engineer(adventure-works\adull)	06.07.1976
9	Research and Development Engineer(adventure-works\adull)	21.02.1965
10	Research and Development Manager(adventure-works\adull)	01.01.1974
11	Senior Tool Designer(adventure-works\adull)	10.02.1907

Fig. 9.1. Result of selection from the InfoEmployees view

Comment. This view restricts access to some fields of the original Employee table, in which case the table is said to have a vertical filter, i.e. access to some of the table fields without column-level protection is restricted.

Task 2: Create a view that provides access to information only about manufacturers that have online services for ordering goods.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

Directions for implementation:

1. By analogy with the previous task, it is necessary to create a vertical filter, i.e. select only one Name field from the Vendor table.
2. To cut off "extra" manufacturers, those for which the address of the online order service is unknown, we will use the WHERE section. In this case, we will also create a horizontal filter.
3. Code to create a view it would be: CREATE VIEW
OnlineVendors AS
SELECT [Name] FROM Purchasing.Vendor
WHERE PurchasingWebServiceURL IS NOT NULL

Task 3. Create a view that displays the total cost of each order, indicating the customer and its number.

Directions for implementation:

1. To get all the information we need, we need to join the Customer, Person, SalesOrderHeader and SalesOrderDetail tables.
2. In order to get the total cost of the order, we need to group the data by the SalesOrderID field. However, we will also need to add the fields that form the customer name to the GROUP BY section, since we will need to specify them in SELECT.
3. To determine the cost of one order, we will create the following formula:
 $\text{UnitPrice} * \text{OrderQty} * (1 - \text{UnitPriceDiscount})$.
4. Resulting The code to create the view will be as follows: CREATE
VIEW InfoOrders AS
SELECT FirstName + ' ' + LastName as [Company Name], SalesOrderHeader.SalesOrderID
as [Order Number],
Convert (money, SUM(UnitPrice*OrderQty*(1-UnitPriceDiscount)),0) AS [Total]
FROM (Person.Person INNER JOIN Sales.Customer
ON Person.BusinessEntityID = Customer.PersonID
INNER JOIN Sales.SalesOrderHeader
ON Customer.CustomerID=SalesOrderHeader.CustomerID)
INNER JOIN Sales.SalesOrderDetail
ON SalesOrderHeader.SalesOrderID=SalesOrderDetail.SalesOrderID
GROUP BY SalesOrderHeader.SalesOrderID, FirstName + ' ' + LastName

Independent work

1. Identify 5 customers who are purchasing leaders (format the result as a stored procedure).

Indication: Use the data from the InfoOrders view.

2. Display statistics for AdventureWorks2008 showing sales volumes and number of orders by month.

Indication: Create a view that contains the order number, the year and month the order was placed, and the total cost of each order with the discount. To get the year and month the order was placed, use the convert function convert (char(7),OrderDate,102), which converts the date to a string of type yyyy.mm.dd. Since the conversion specifies a string of 7 characters, the result of the function will be a string of type yyyy.mm.

3. Create an InfoSalesPerson view that displays salesperson statistics for AdventureWorks2008: the view should include the total amount of orders received and the number of orders received.
4. Determine the number of orders placed by season.

Microsoft SQL Server. Lab #10. Database Design

Objective: demonstrate the database design process and teach how to use it SQL commands for creating tables and managing constraints.

Theoretical material: Before performing the lab work, it is recommended to study Lecture No. 10 “First Normal Forms” and Lecture No. 11 “Fourth and Fifth Normal Forms”, which discuss the need for and the process of normalization itself, which allows maintaining the integrity of the database.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

Reporting requirements: based on the results of the work, provide a screenshot with the physical data model, as well as screenshots with SQL commands and the results of their execution for tasks No. 2 and No. 3 from the section "Independent work".

Database Design Stages

From the Microsoft perspective, database design consists of two phases: logical and physical modeling. During logical design, a database model is developed that is independent of a specific DBMS. During physical design, a database model is created that is optimized for a real DBMS.

The database design process consists of the following steps:

1. Collection of information.
2. Identification of objects and their modeling.
3. Defining the types of information for each object.
4. Relationship identification.
5. Normalization.
6. Creating a database.

The first five stages belong to the logical modeling phase, and the sixth stage belongs to the physical modeling phase.

Logical model

We will develop a logical structure for a database that stores information about teachers and the courses they teach, information about students and the classes they attend.

It has been established that:

- a student can attend any number of courses;
- teachers can teach several subjects;
- the course is taught to a certain audience;
- Only one subject is taught in one classroom.

Thus, the following objects are distinguished: Teachers, Students, Disciplines and Classrooms. The following relationships exist between the objects: Students–Disciplines (many-to-many, implemented through the additional object Registration), Teachers–Disciplines (one-to-many), Classrooms–Disciplines (one-to-one).

Each object must contain the following information.

Teachers:

- surname, first name, patronymic – mandatory attribute;
- date of birth is a mandatory attribute, it cannot be greater than today's date and less than 1900;
- home address is a required attribute; if not specified, the default value is 'unknown';
- phone – a mandatory attribute, either specified in the format 00-00-00, or if it is absent, 'no' is specified by default.
- hire date is a mandatory attribute, cannot be greater than today's date and less than the date of birth, today's date is set by default;
- Experience is a calculated field, determined by the difference between today's date and the hire date.

Students:

- surname, first name, patronymic – mandatory attribute;
- date of birth is a mandatory attribute, it cannot be greater than today's date and less than 1900;

Disciplines:

- the name of the discipline is a mandatory attribute;
- the name of the teacher teaching the course, the list of students attending the course, the audience – foreign key.

Audiences:

- number is a mandatory attribute;
- audience type is a required attribute.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

Physical model

When moving from a logical model to a physical one, it is necessary to ensure data integrity, which is understood as the consistency and correctness of the information in the database.

There are 4 groups of integrity rules:

- Value domain (or field) integrity: defines the set of valid values for a field, including whether NULL values are allowed.
- Entity integrity: requires that each table record have a unique identifier – the primary key value.
- Referential integrity: ensures that the relationship between the primary key and the foreign key is maintained at all times, i.e. it prohibits deleting a record or changing its primary key value if it is referenced by a foreign key, and it prohibits adding a record with a foreign key value that does not correspond to any primary key value.
- User-defined integrity specifies user-specific rules that cannot be assigned to any group.

Data integrity can be ensured in two ways.

1. **Declarative method.** The criteria that the data must satisfy are specified when the object is defined and are part of the database definition.

Advantages: Integrity control is performed automatically by MS SQL Server, this method is fully compatible with the ISO SQL:2008 standard.

2. **Procedural method.** The criteria are described in operator packages, the execution of which determines the integrity of the data.

Advantages: implemented both on the client and on the server using various software tools.

Declarative integrity

It is implemented using constraints that are the most recommended for ensuring data integrity. Each type of integrity is ensured by a corresponding constraint:

- PRIMARY KEY. Defining a field or group of fields as a primary key allows you to uniquely identify each record in the table, since duplicate and NULL values are not allowed.
- FOREIGN KEY. Allows you to establish a relationship between fields containing identical data. The data in this field can take the values defined in the corresponding primary key or NULL values.
- UNIQUE. Defines the uniqueness of data in a certain field of a table, and this constraint allows NULL values. In one table, the use of several such constraints, fields with such a constraint can be used as foreign keys.
- DEFAULT. Specifies the default value for the field if a value is not explicitly specified when inserting data. Only one such constraint can be applied to a field.
- CHECK. Imposes a constraint on the field value in the form of a logical expression, which allows you to determine the range of acceptable field values. Moreover, this constraint allows you to refer to the values of other fields.

To obtain information about the restrictions used by the specified table, a special stored procedure is used:

sp_helpconstraint*TableName*

Creating a table

For example, to create a table storing information about students, you need to run the command:

```
CREATE TABLE Students
(StudentID int IDENTITY(1,1)
CONSTRAINT prkStudentID PRIMARY KEY,
LastName char(20) NULL,
FirstName char(20) NULL,
BirthDate datetime NULL
CONSTRAINT bdChck CHECK (BirthDate < GetDate()) )
```

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

To obtain information about the properties of a table, you need to execute a special stored procedure: `sp_help TableName`.

Projections of the information scheme

System information about the database and all its objects is called metadata, for obtaining which MS SQL Server provides many different ways: calling system stored procedures and information schema views, as well as using the SQL Management Studio administration utility.

Information schema views provide an external view of metadata independent of the system tables. These views are owned by a predefined user, `information_schema`. Each view contains metadata for objects in a database.

For example, the following views return metadata about the current database constraints:

- `table_constraints` – list of all database constraints.
- `check_constraints` – list of CHECK type constraints.
- `referential_constraints` – list of referential integrity constraints.

Independent work

1. Based on the described logical model of the database, create its physical model.
2. Define metadata of the PRIMARY KEY constraints in the created database, specifying the names of the constraints and the tables to which they apply.
3. Output logical expressions of CHECK constraints.

Microsoft SQL Server. Lab #11. Data Management

Target: teach how to use SQL statements to insert, modify, and delete data from tables.

Theoretical material: Before performing the lab, it is recommended to study Lecture #4 “Data Selection”, which requires knowledge of the SELECT operator, which is necessary for inserting data into one table from another, as well as Lecture #9 “Relational Algebra Operators”, which covers the theoretical aspects of the basic data management commands.

Reporting requirements: based on the results of the work, submit a set of SQL scripts that solve the problems from the "Independent Work" section, as well as a text file required for the task No. 2 from the same section.

Task 1. Add a record to the Lect table with the specified field values and default values.

Instructions for implementation:

1. Let's set values for three fields: `Brdate`, `FirstName`, `LastName`.
2. We will not specify the counter field, since it will be generated automatically.
3. Use the following code to insert a new record: `INSERT INTO Lect (Brdate, FirstName, LastName) VALUES (Convert(datetime,'01.01.1970',104), '...', '...')`

Comment. Running the `INSERT INTO Lect DEFAULT VALUES` command will result in an error because not all fields have default values specified.

Task 2. Add data from the Employee table of the database to the Lect table

AdventureWorks2008:

Directions for implementation:

1. We will fill in 4 fields in the Lect table: `FirstName`, `LastName`, `Brdate`, `HireDate`.
2. For To insert several records at once, use the SELECT operator: `INSERT INTO Lect (FirstName, LastName, Brdate, HireDate) SELECT FirstName, LastName, BirthDate, HireDate FROM AdventureWorks2008.HumanResources.Employee INNER JOIN AdventureWorks2008.Person.Contact ON Contact.ContactID = Employee.ContactID`

Comment. When using the SELECT command, multiple records are inserted into a table,

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

that are the result of selecting data from other tables. However, if even one of the inserted records violates the integrity constraints of the table, the entire INSERT command is canceled.

Task 3. Add data from the file to the Students table.

Directions for implementation:

1. Create a text file students.txt in the root of drive C: with the following contents:

```
1 Ivan          Petrov1 . 1 . 1 9 9 0
2 Petr         Ivanov1 2 . 1 0 . 1 9 9 2
3 Sergey       Kazakov5.3.1991
```

Comment. TAB is used as a line separator instead of spaces.

2. Run the following code:

```
BULK INSERT Students FROM 'c:\students.txt'
```

Comment. Note that although we specified the StudentID fields in the students.txt file, they were ignored during the insert. This is because we did not use the KEEPIDENTITY option.

Task 4. Change the phone number of a teacher named King.

Directions for implementation:

1. To find the desired name, we will use the LIKE operator.
2. Let's install it for all teachers 'King' phone equal 35-35-35: UPDATE

```
Lect
SET Phone = '35-35-35'
WHERE FirstName like 'King'
```

Task 5. Specify that computer science courses should be taught by a teacher named King.

Instructions for implementation:

1. To update records in the Course table, we need to use a subquery that returns the desired teacher: SELECT LectID From Lect Where (FirstName like 'King').
2. It is important to remember that the subquery must return a single value for each record.
3. Code to update the computer science teacher it will look like this: UPDATE Course SET

```
LectID =
(SELECT LectID From Lect Where (FirstName like 'King'))
WHERE CourseName like '%[Ii]nform%'
```

Task 6. Remove all students with the last name Ivanov.

Directions for implementation:

1. To delete records based on a given condition, you must use the WHERE clause in the DELETE statement.
2. Do it the following code to delete students:

```
DELETE Students
WHERE LastName = 'Ivanov'
```

Comment. Removing data from a table is performed row by row, and it is possible to delete either one record or several records that satisfy a certain condition. However, to delete all records of a certain table, the following command is recommended: TRUNCATE TABLE TableName. In this case, all records are deleted, but the table structure is preserved, as are the objects associated with it. This command is executed faster than the DELETE command, since the server only registers the release of memory pages, and not the deletion of each record from the page.

Independent work

1. Fill in the Lect and Class tables of the created database.
2. Fill in the Students table of the created database from the text file.
3. Fill in the remaining tables of the created database.

Remove all students born before 1990.

Tasks for independent work of the student and methodical recommendations for their fulfillment

Relational databases (RDBs) are the foundation for storing and managing data in modern information systems. Through independent work, students will deepen their knowledge of RDB design, implementation, and management.

Independent work of students				
Week №	Module №	The theme of the independent work of students	Number of hours	Types and methods of training
1	1	Database architectures	9	Report
2	1	Types of Database Management	9	Report
3	1	Database design	9	Essay
4	1	Hierarchical data model	9	Report
5	1	Conceptual model of a database	9	Essay
6	1	Basic concepts of database systems	9	Report
7	1	Scaling Distributed Databases	9	Essay
8	2	Data flow management	9	Report
9	2	Data warehouses	9	Report
10	2	Distributed and parallel databases	9	Report
11	2	SQL language	9	Report
12	2	XQuery language	9	Report
13	2	NoSQL language	9	Report
14	2	OLAP technologies	9	Essay
15	2	Types of modern databases	9	Essay
TOTAL			135	

Independent Work Structure

1. Work Topic: All topics are listed in the table.
2. Work Objectives: Define your goals and what you want to achieve during the work.
3. Theoretical Section: Prepare a theoretical study based on the type specified in column 5 of the table.
4. Practical Section, if required by the topic:
 - Database Design:
 - Define entities and their attributes.
 - Create an entity-relationship diagram (ER diagram).
 - Normalize the database to third normal form (3NF).
 - Database Creation:
 - Use a DBMS (e.g., MySQL, PostgreSQL) to create the database.
 - Write SQL scripts to create tables and define relationships between them.
 - Database Population:
 - Prepare test data and load it into the database using SQL queries.

IWS #1

Research the similarities and differences of approaches, provide implementation examples
Prepare an essay.

To prepare an essay, you must select literature, learn to read it correctly, and take notes. The goal is to extract the necessary information from the text. The essay should be innovative, reflect a current view on specific issues, and may be supplemented with statistical data.

IWS #2

Support for object extensions in PostgreSQL and Oracle: Create a mini-research and design a presentation.

In the mini-research, the student must reflect the area of interest explored in the work, its distinctive features from existing similar publications, the strengths and weaknesses of the work, the author's contribution to the development of the problems under study and the breadth of their coverage, the originality of ideas, approaches, and style of presentation. A presentation should also be prepared, including cartographic materials, diagrams, graphs, audio, photographs, drawings, and other materials.

IWS #3

Standards and Their Support in Popular DBMSs Prepare an Essay

This is a longer form of independent student work than an essay, containing information that complements and develops the main topic covered in class. An essay may include a review of several sources and serve as the basis for a presentation on a specific topic at seminars or conferences.

Before writing an essay, it is necessary to create a plan and highlight the following sections: Introduction, Main Body, Conclusion, and a list of references. Essays can be divided into two main groups (types): scientific-problem-based and overview-based. Essay length is 10-12 pages with a 1.0 line spacing.

IWS #4

Development of an Object Schema for a Subject Area (e.g., a Booking System) Practical Assignment

This practical assignment is designed to assess theoretical knowledge and the ability to apply database development algorithms.

IWS #5

Inheritance and Polymorphism in Database Object Types Prepare an Essay

This is a longer form of independent student work than an essay, containing information that complements and develops the main topic covered in class. An essay may include a review of several sources and serve as the basis for a presentation on a specific topic at seminars or conferences.

Before writing an essay, it is necessary to create a plan and highlight the following sections: Introduction, Main Body, Conclusion, and a list of references. Essays can be divided into two main groups (types): scientific-problem-based and overview-based. Essays should be 10-12 pages long with a 1.0 line spacing.

IWS #6

Business Logic through Object Methods Practical Assignment

This practical assignment is designed to assess theoretical knowledge and the ability to apply database development algorithms. SRO #7

Converting UML Class Diagrams into Object-Relational Structures Practical Assignment

This practical assignment is designed to assess theoretical knowledge and the ability to apply database development algorithms.

IWS #7

Transforming UML Class Diagrams into Object-Relational Structures (Practical Task)

This practical task is designed to assess theoretical knowledge and the ability to apply database development algorithms.

IWS #8

Investigating Problems Arising When Using Objects in a Relational DBMS (Mini-Study)

In the mini-study, the student must reflect the area of interest explored in this work, its distinguishing features from existing similar publications, the strengths and weaknesses of the work, the author's contribution to the development of the problems under study and the breadth of their coverage, the originality of ideas, approaches, and writing style.

IWS #9

Creating Triggers for Objects: Automating Database Behavior (Practical Task)

This practical task is designed to assess theoretical knowledge and the ability to apply database development algorithms.

IWS #10

Organizing Access to Nested Objects in SQL Queries Prepare an Essay

This is a longer form of independent student work than an essay, containing information that complements and develops the main topic covered in class. An essay may include a review of several sources and serve as the basis for a presentation on a specific topic at seminars or conferences.

Before writing an essay, it is necessary to create a plan and highlight the following sections: Introduction, Main Body, Conclusion, and a list of references. Essays can be divided into two main groups (types): scientific-problem-based and overview-based. Essays should be 10-12 pages long with a 1.0 line spacing.

IWS #11

Stress Testing of Standard and Object-Based Schemas Prepare an Essay

This is a longer form of independent student work than an essay, containing information that complements and develops the main topic covered in class. An abstract can include a review of several sources and serve as the basis for a presentation on a specific topic at seminars or conferences.

Before writing your abstract, it's necessary to create a plan and highlight the following sections: Introduction, Main Body, Conclusion, and a list of references. Abstracts can be divided into two main groups (types): scientific-focused and general-informational. The length should be between 10-12 pages, with a 1.0-point spacing.

IWS #12

Object-Relational Capabilities in IBM Db2 and MS SQL Server Mini-Research

In the mini-research, the student must reflect the area of interest explored in the work, its distinguishing features from existing similar publications, the strengths and weaknesses of the work, the author's contribution to the development of the problems under study and the breadth of their coverage, the originality of ideas, approaches, and writing style.

IWS #13

ORM Approaches (SQLAlchemy, Hibernate) and Direct Object Calling Prepare an Essay

This is a more comprehensive form of independent student work than an essay, containing information that complements and develops the main topic covered in the classroom. The essay may include a review of several sources and serve as the basis for a presentation on a specific topic at seminars or conferences.

Before writing the essay, it is necessary to create an outline and highlight the following sections: Introduction, Main Body, Conclusion, and a list of references. Abstracts can be divided into two main groups (types): scientific-problem-based and overview-based. They are 10-12 pages long with a 1.0-point spacing.

IWS #14

Preparing a migration plan, identifying potential difficulties. Practical assignment

This practical assignment is designed to assess theoretical knowledge and the ability to apply database development algorithms.

IWS #15

Developing a mini-project: object-relational database model for an online store/library/CRM

Practical assignment

This practical assignment is designed to assess theoretical knowledge and the ability to apply database development algorithms.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

MATERIALS FOR CURRENT, MIDTERM CONTROL, INTERMEDIATE CERTIFICATION

Midterm 1:

1. Describe the database architecture.
2. Data independence.
3. Data bank components.
4. What is a data dictionary?
5. How is the database managed?
6. conceptual model of the subject area.
7. logical model.
8. physical model.
9. stages of database design.
10. Describe the essence of the Infological model.
11. Describe the Entity-Link model.
12. What is a physical model?
13. Data structure diagram. Hierarchical data model.
14. relational databases.
15. data types, domain, access scheme, database scheme.
16. What is relational computing?
17. describe 1NF.
18. Describe 2NF.
19. Describe 3NF.
20. Describe the Boyce-Codd normal form.
21. What is relational algebra? Name the main properties of MS SQL Server MS SQL Server
22. characterize the structure of SQL.
23. main groups of SQL operators.
24. what is included in SQL.
25. What is Dynamic SQL.
26. MS modification operators.
27. object-oriented database architecture.
28. What is a Transaction?
29. What data types are there in MySQL?
30. SELECT query syntax. Single table questions.

Midterm 2:

31. where is the offer
32. Checking collection membership.
33. checking for NULL value.
34. checking two values for difference.
35. complex search conditions.
36. string functions.
37. numerical functions.
38. functions of day and time.
39. transformation function.
40. aggregate functions.
41. sorting query results.
42. internal connection connection state.
43. OUTER JOIN relationship state.
44. LEFT JOIN connection condition.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

45. connection condition: correct connection.
46. condition of full accession.
47. using the application design.
48. Cartesian products.
49. accession.
50. natural connection.theta connection.
51. merge a table with its copy.
52. simple subqueries in the WITH clause.
53. simple subqueries in the SELECT clause.
54. simple subqueries in the FROM clause.
55. simple subqueries in the WHERE, HAVING clause.
57. related subqueries in the SELECT clause.
58. related queries in the "WHERE" clause, "HAVING".
59. recursive queries in the WITH branch.
60. FIRST.. What functions do the SKIP, ROWS structures perform?

Final Control:

1. Describe the database architecture.
2. Data independence.
3. Data bank components.
4. What is a data dictionary?
5. How is the database managed?
6. conceptual model of the subject area.
7. logical model. 8.physical model.
9. stages of database design.
10. Describe the essence of the Infological model.
11. Describe the Entity-Link model.
12. What is a physical model?
13. Data structure diagram. Hierarchical data model. 14.relational databases.
15. data types, domain, access scheme, database scheme.
16. What is relational computing?
17. describe 1NF.
18. Describe 2NF.
19. Describe 3NF.
20. Describe the Boyce-Codd normal form.
21. What is relational algebra? Name the main properties of MS SQL Server MS SQL Server
22. characterize the structure of SQL.
23. main groups of SQL operators.
24. what is included in SQL.
25. What is Dynamic SQL.
26. MS modification operators.
27. object-oriented database architecture.
28. What is a Transaction?
29. What data types are there in MySQL?
30. SELECT query syntax. Single table questions.
31. where is the offer
32. Checking collection membership.
33. checking for NULL value.
34. checking two values for difference.

	L.N. Gumilyov Eurasian National University	Educational and methodological complex of the discipline	Edition: seventh
---	--	--	------------------

35. complex search conditions.
36. string functions.
37. numerical functions.
38. functions of day and time.
39. transformation function.
40. aggregate functions.
41. sorting query results.
42. internal connection connection state.
43. OUTER JOIN relationship state.
44. LEFT JOIN connection condition.
45. connection condition: correct connection.
46. condition of full accession.
47. using the application design.
48. Cartesian products.
49. accession.
50. natural connection.theta connection.
51. merge a table with its copy.
52. simple subqueries in the WITH clause.
53. simple subqueries in the SELECT clause.
54. simple subqueries in the FROM clause.
55. simple subqueries in the WHERE, HAVING clause. 57.related subqueries in the SELECT clause.
58. related queries in the "WHERE" clause, "HAVING".
59. recursive queries in the WITH branch.
60. FIRST.. What functions do the SKIP, ROWS structures perform?